Data Structures & Analysis Of Algorithm Laboratory
[Simulation and Analysis]

Dr. Bibhudatta Sahoo

Communication & Computing Group

Department of CSE, NIT Rourkela

Email: bdsahu@nitrkl.ac.in, 9937324437, 2462358

Algorithm

- Algorithm: is a procedure that consists of a ***finite set of instructions*** which, given an ***input*** from some set of possible inputs, enables us to obtain an *output* if such an output exists or else obtain nothing at all if there is no output for that particular input through a ***systematic execution*** of the ***instructions.***

- An algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.

- As a computer professional, it is useful to know a standard set of important algorithms and to be able to design new algorithms as well as to analyze their efficiency.

Algorithm

- **A clearly specified set of instructions to solve a problem.**
- Characteristics:
  - **Input:** Zero or more quantities are externally supplied
  - **Definiteness:** Each instruction is clear and unambiguous
  - **Finiteness:** The algorithm terminates in a finite number of steps.
  - **Effectiveness:** Each instruction must be primitive and feasible
  - **Output**: At least one quantity is produced

Design and Analysis of Algorithms

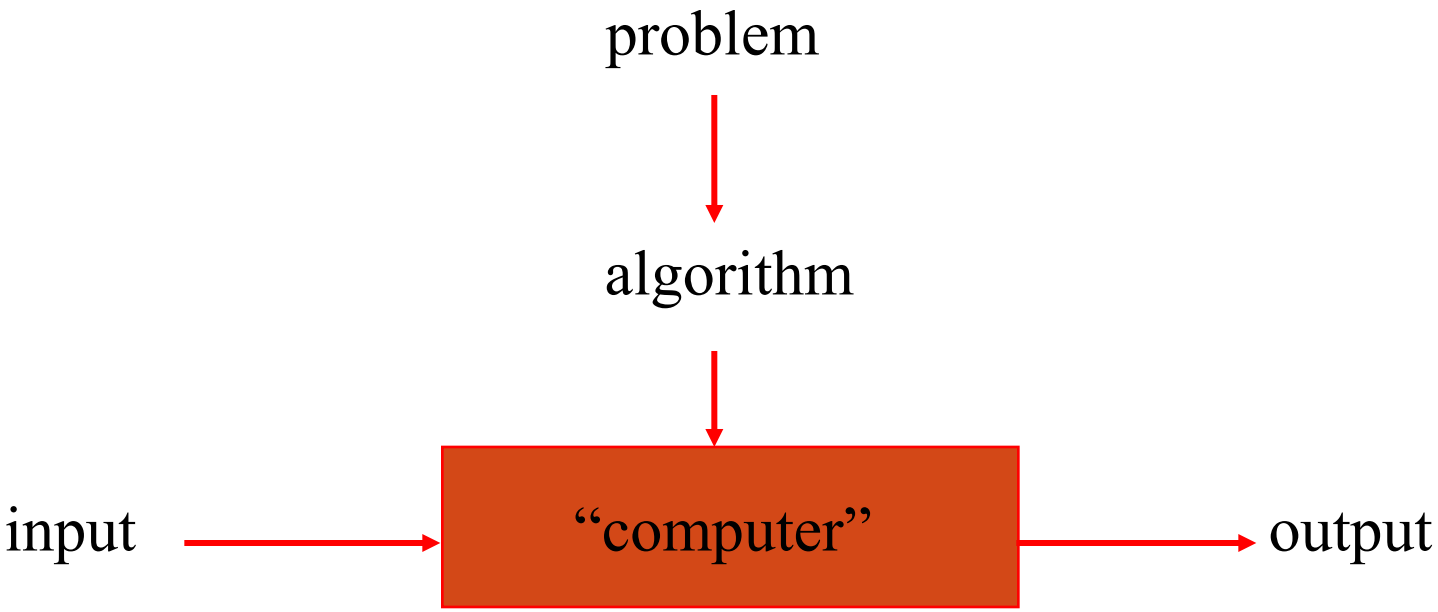
- ***Analysis:*** predict the cost of an algorithm in terms of resources and performance

- ***Design:*** design algorithms which minimize the cost

Analysis of algorithms

- How good is the algorithm?
  - Correctness
  - **Time efficiency**
  - **Space efficiency**
  - Simplicity
  - Generality
- Does there exist a better algorithm?
  - Lower bounds
  - Optimality

Notion of algorithm



Algorithmic solution

Problem

- An ***instance of a problem*** consists of all inputs needed to compute a solution to the problem.
- An algorithm is said to be ***correct*** if for every input instance, it halts with the correct output.
- A **correct algorithm** ***solves*** the given computational problem.
- An **incorrect algorithm** might not halt at all on some input instance, or it might halt with other than the desired answer.

Why Analyze Algorithms?

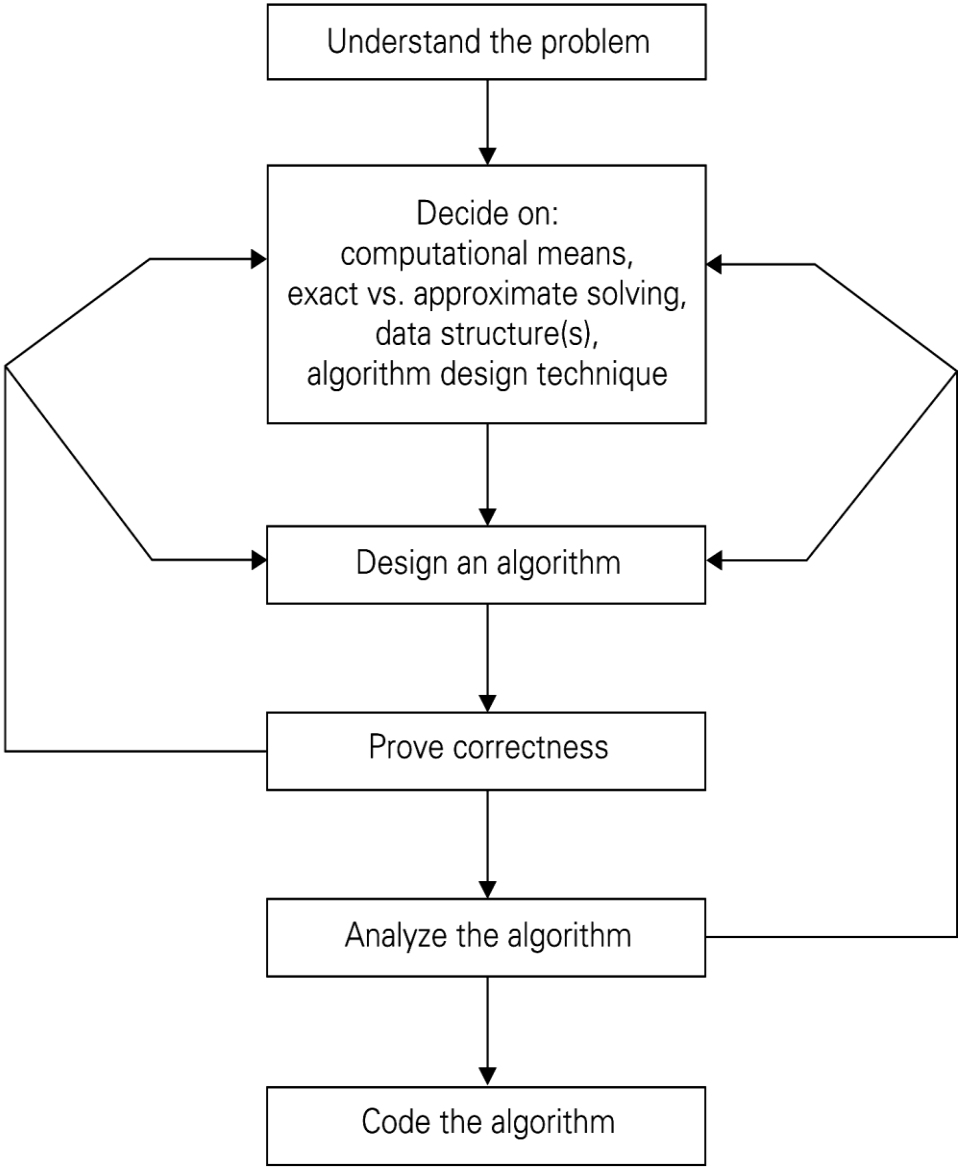
An algorithm can be analyzed in terms of time efficiency or space utilization. We will consider only the former right now. The running time of an algorithm is influenced by several factors:

- Speed of the machine running the program
- Language in which the program was written. For example, programs written in assembly language generally run faster than those written in C or C++, which in turn tend to run faster than those written in Java.
- Efficiency of the compiler that created the program
- The size of the input: processing 1000 records will take more time than processing 10 records.
- Organization of the input: if the item we are searching for is at the top of the list, it will take less time to find it than if it is at the bottom.

Why study algorithms and performance?

- Algorithms help us to understand ***scalability.***
- Performance often draws the line between what is feasible and what is impossible.
- Algorithmic mathematics provides a ***language*** *for talking about program behavior.*
- Performance is the ***currency*** *of computing.*
- The lessons of program performance generalize to other computing resources.
- Speed is fun!

Algorithm Design and Analysis Process

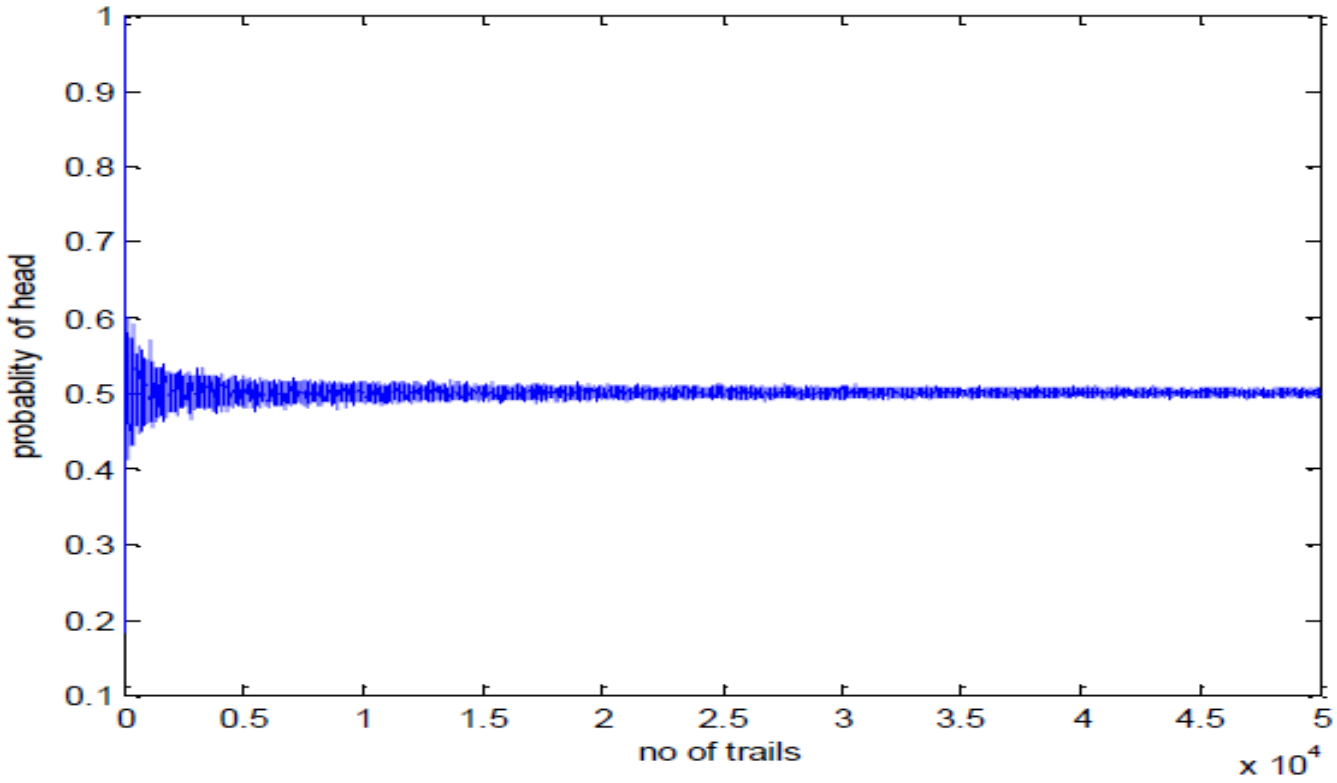


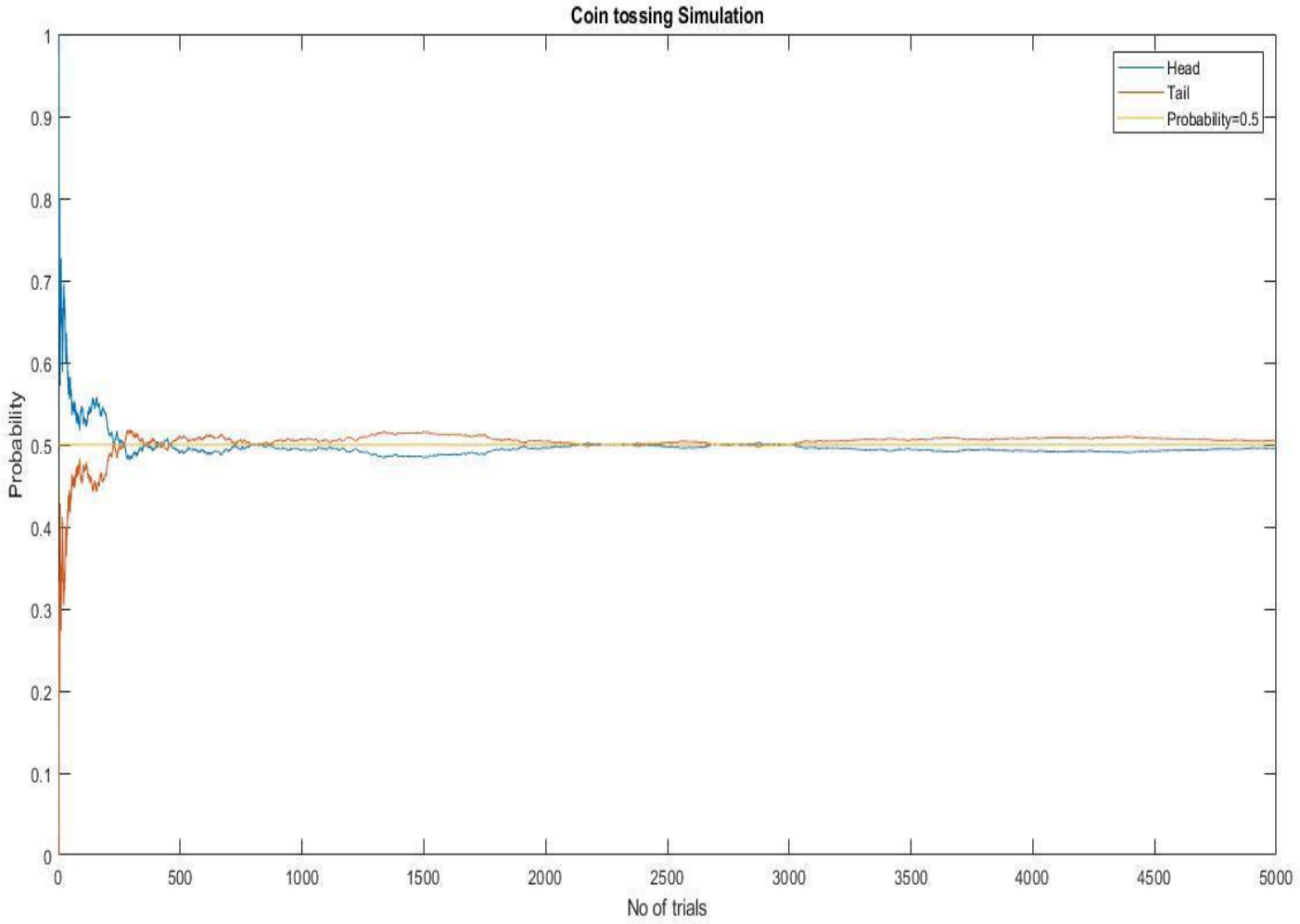
Assignment 1

Q.1. Coin Tossing

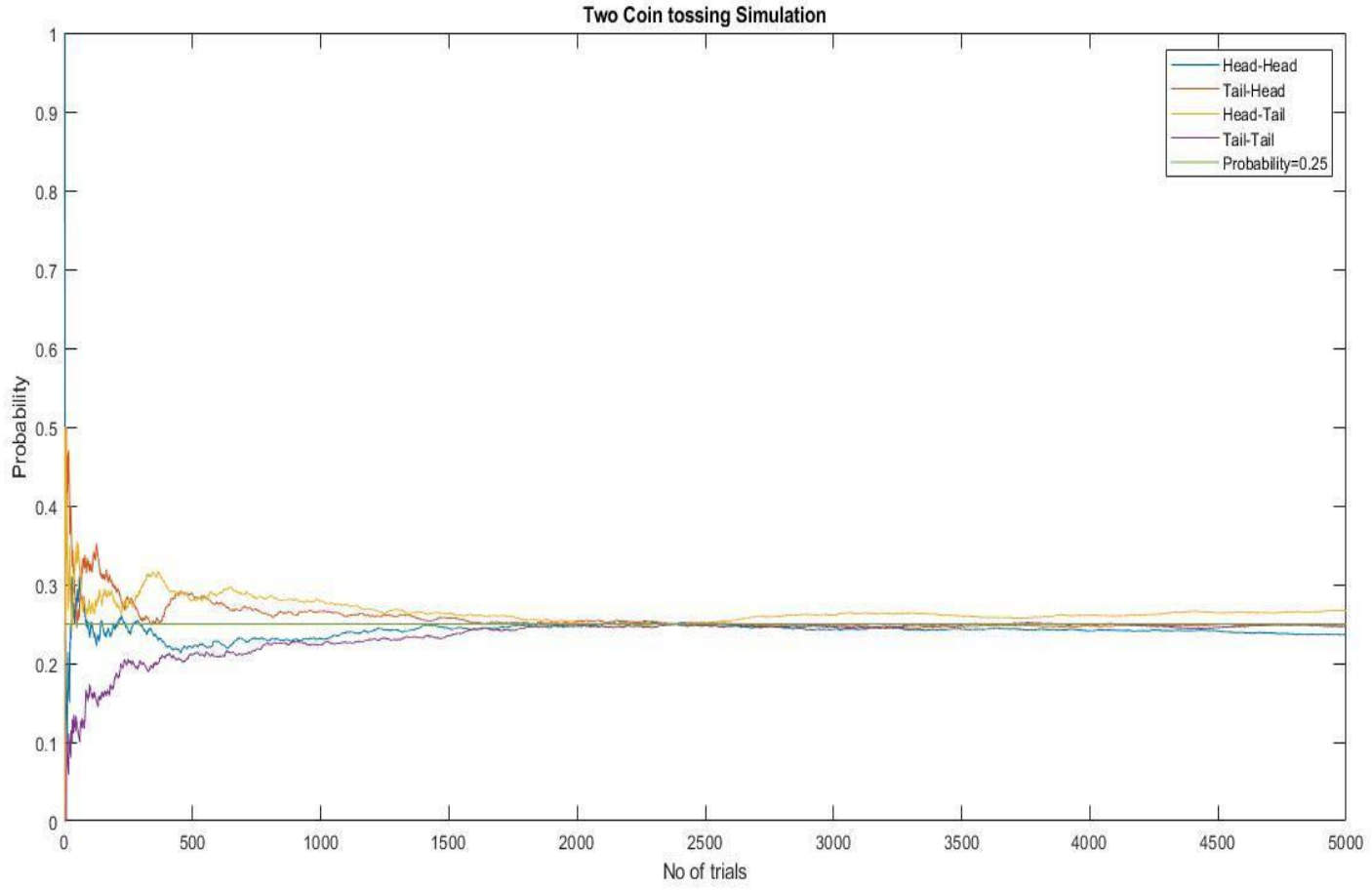
Through the simulation, show that probability of getting HEAD by tossing a fair coin is about 0.5. Write your observation from the simulation run.

Through the simulation, show that probability of getting HEAD by toss coin is about 0.5. Write your observation from the simulation run.




Coin tossing Simulation
Head
Tail
Probability=0.5
Probability
No of trials
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
0
500
1000
1500
2000
2500
3000
3500
4000
4500
5000

Observation: Thus we see that the graph converges to the line y=0.25. So the probability of getting head-head or tail-head or head-tail or tail-tail is 0.25.



Sample Matlab Codes for plotting

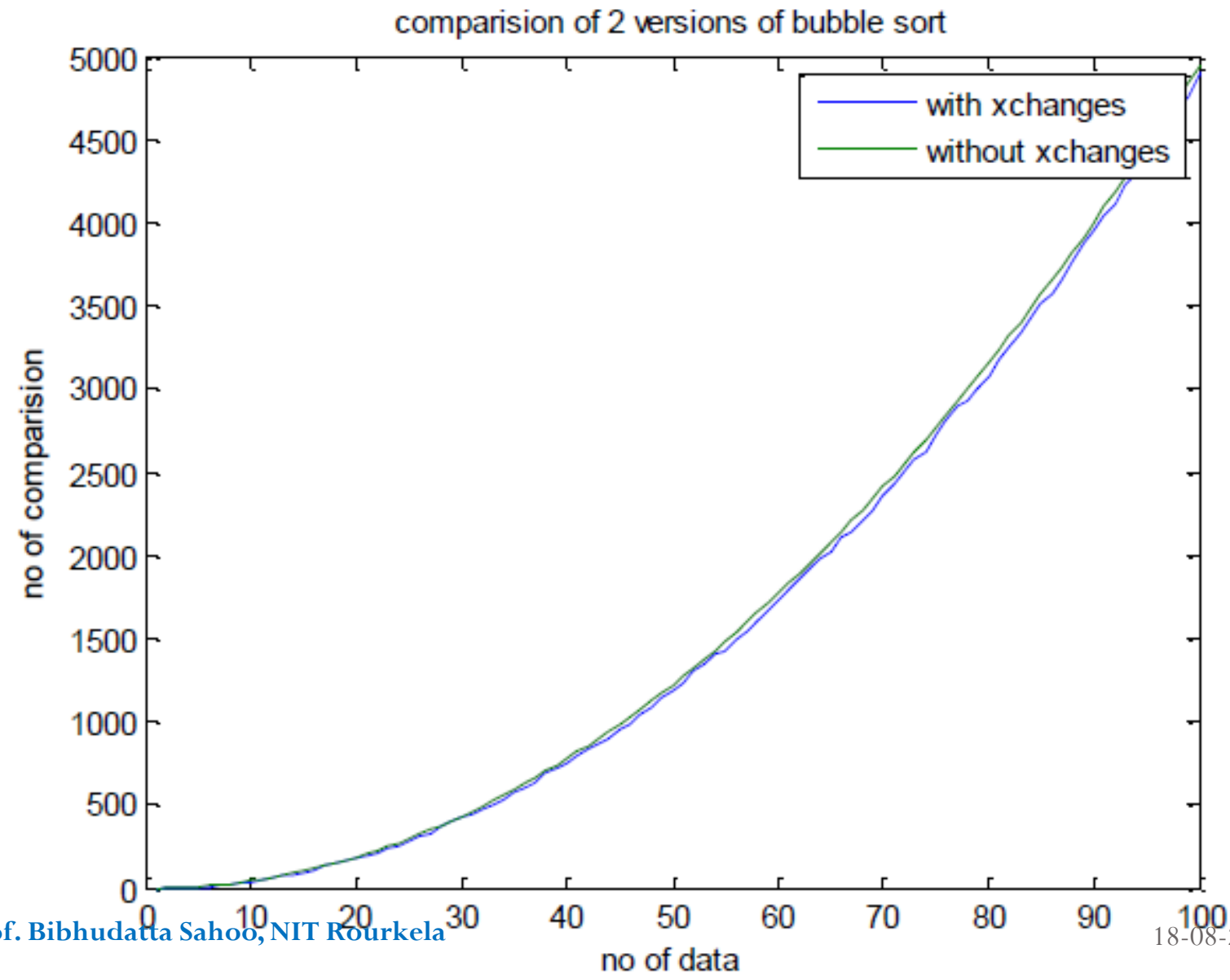
```
plot (xdata, y1data, xdata, y2data);
title('Performance analysis of Sorting')
xlabel('number of elements');
ylabel('number of comparisions');
legend('Bubble Sort', 'Insertion Sort')
grid on;
-----------------------------------------------
disp(' Analyzing the performance of sorting')
```

Question

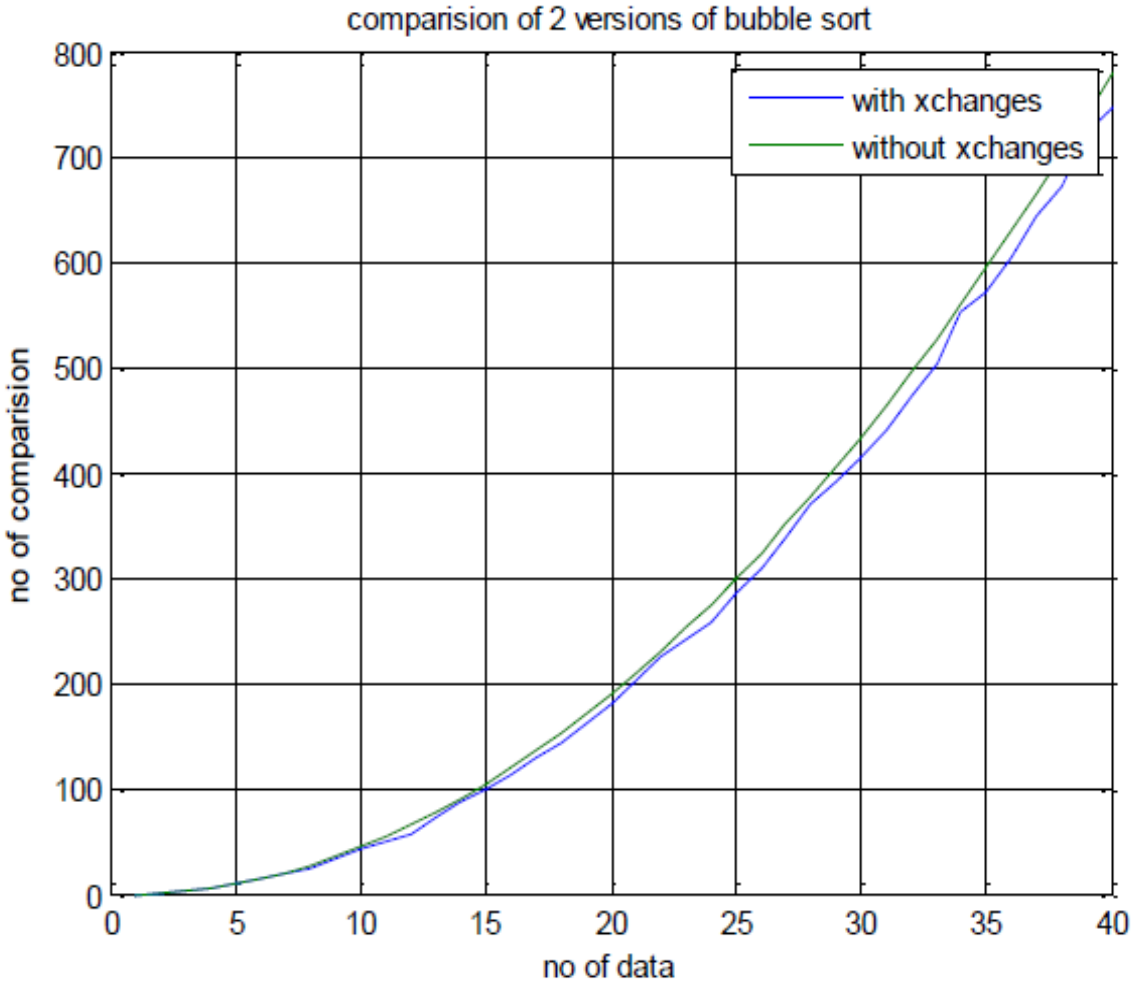
Q.2. Performance analysis of Bubble Sort

Write the program to implement two different versions of bubble sort(BUBBLE SORT that terminates if the array is sorted before n-1^{th} Pass. Vs. BUBBLE SORT that always completes the n-1^{th} Pass) for randomized data sequence.

Performance of Bubble sort



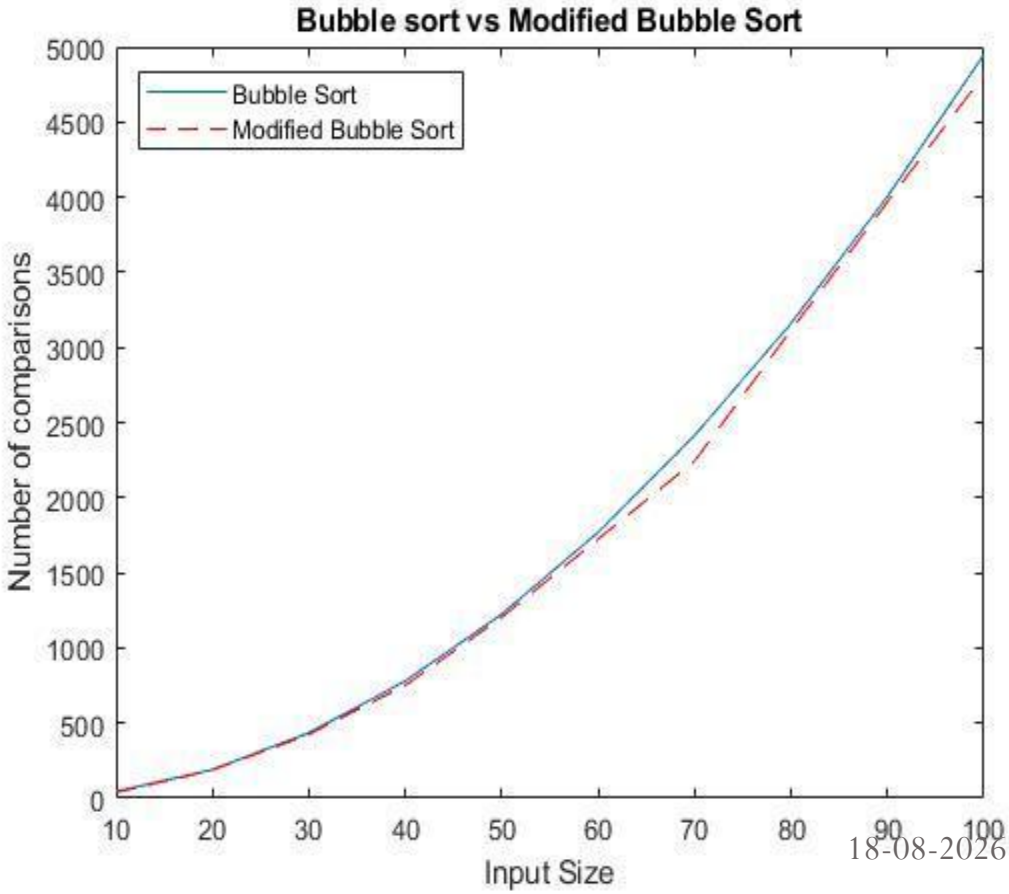
Performance of Bubble sort



Comments : Bubble sort with exchanges taken into consideration works better than bubble sort without exchanges for random data.

Performance of Bubble sort

- Observaton: We can see that the modified bubble sort is better than the bubble sort as it takes less number of comparisons. This is because the loop breaks after the array is sorted but in normal bubble sort the loop continues until n(n+1)/2 comparisons is done.

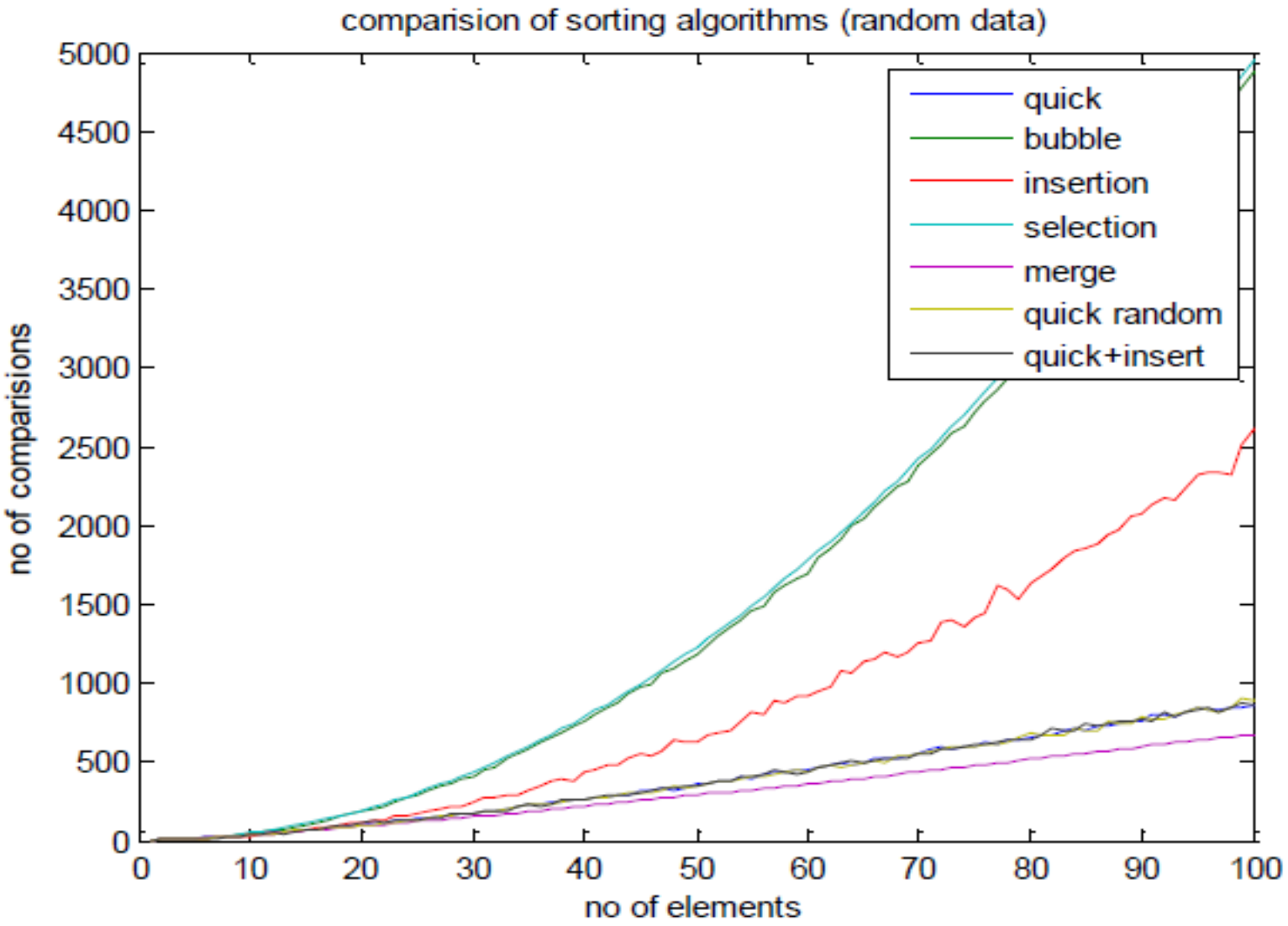


Question

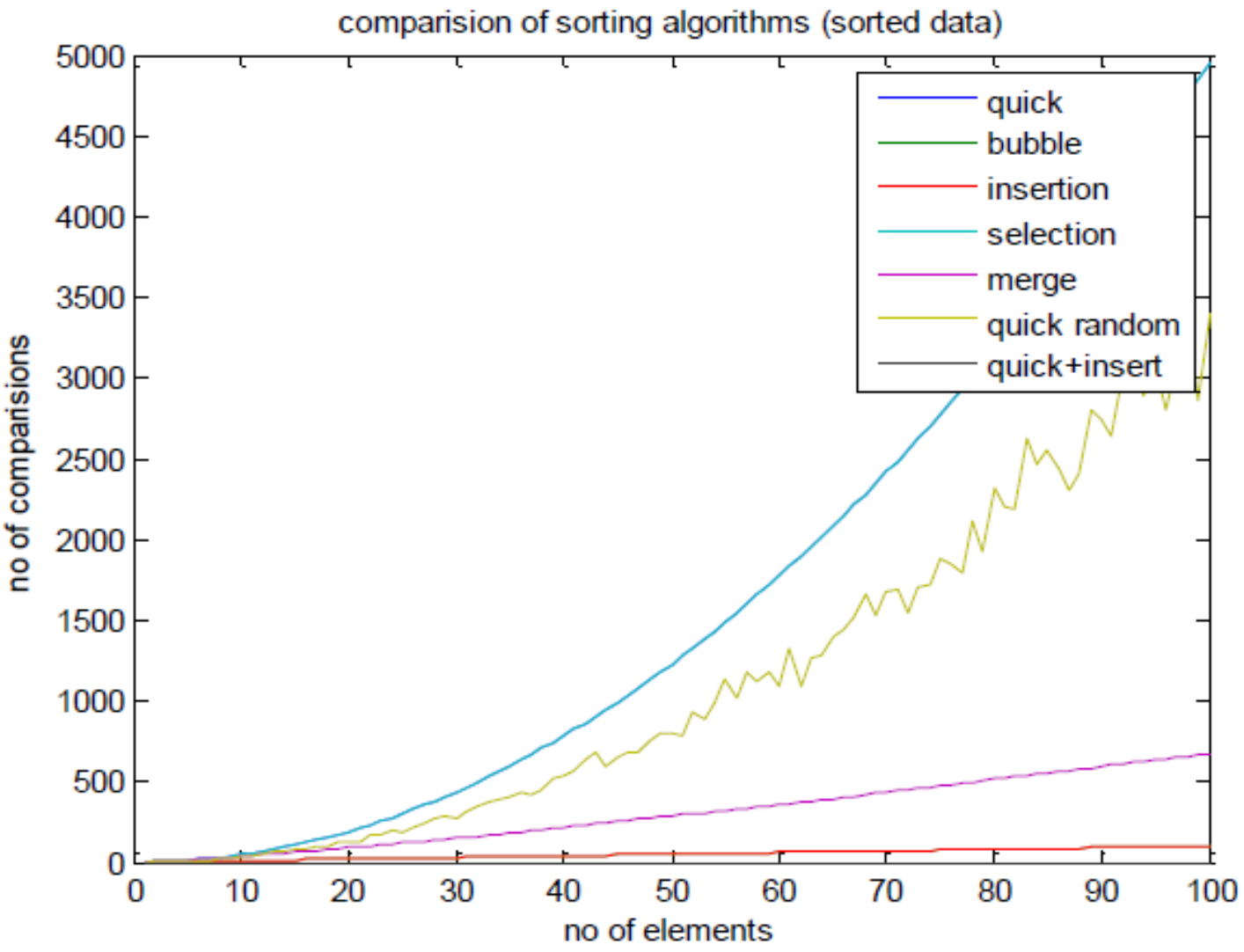
Q.3. Average case analysis for Sorting Algorithms

For each of the data formats: random, reverse ordered, and nearly sorted, run your program say **SORTTEST** for all combinations of sorting algorithms and data sizes and complete each of the following tables. When you have completed the tables, analyze your data and determine the asymptotic behavior of each of the sorting algorithms for each of the data types *(i)* ***Random data, (ii) Reverse Ordered Data, (iii) Almost Sorted Data*** **and (iv)** ***Highly Repetitive Data*** . select the suitable no of elements for the analysis that supports your program.

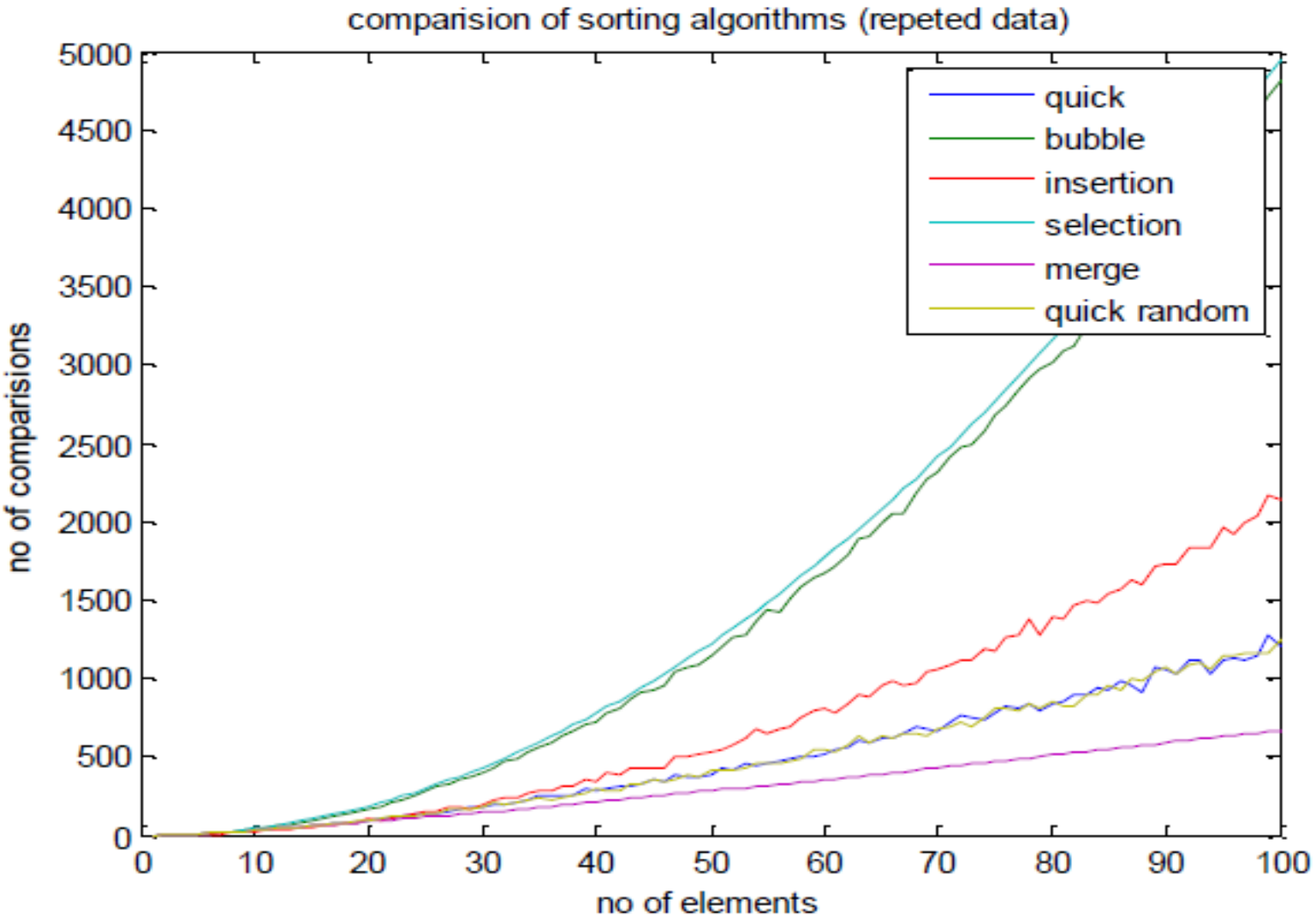
Average case analysis for Sorting Algorithms



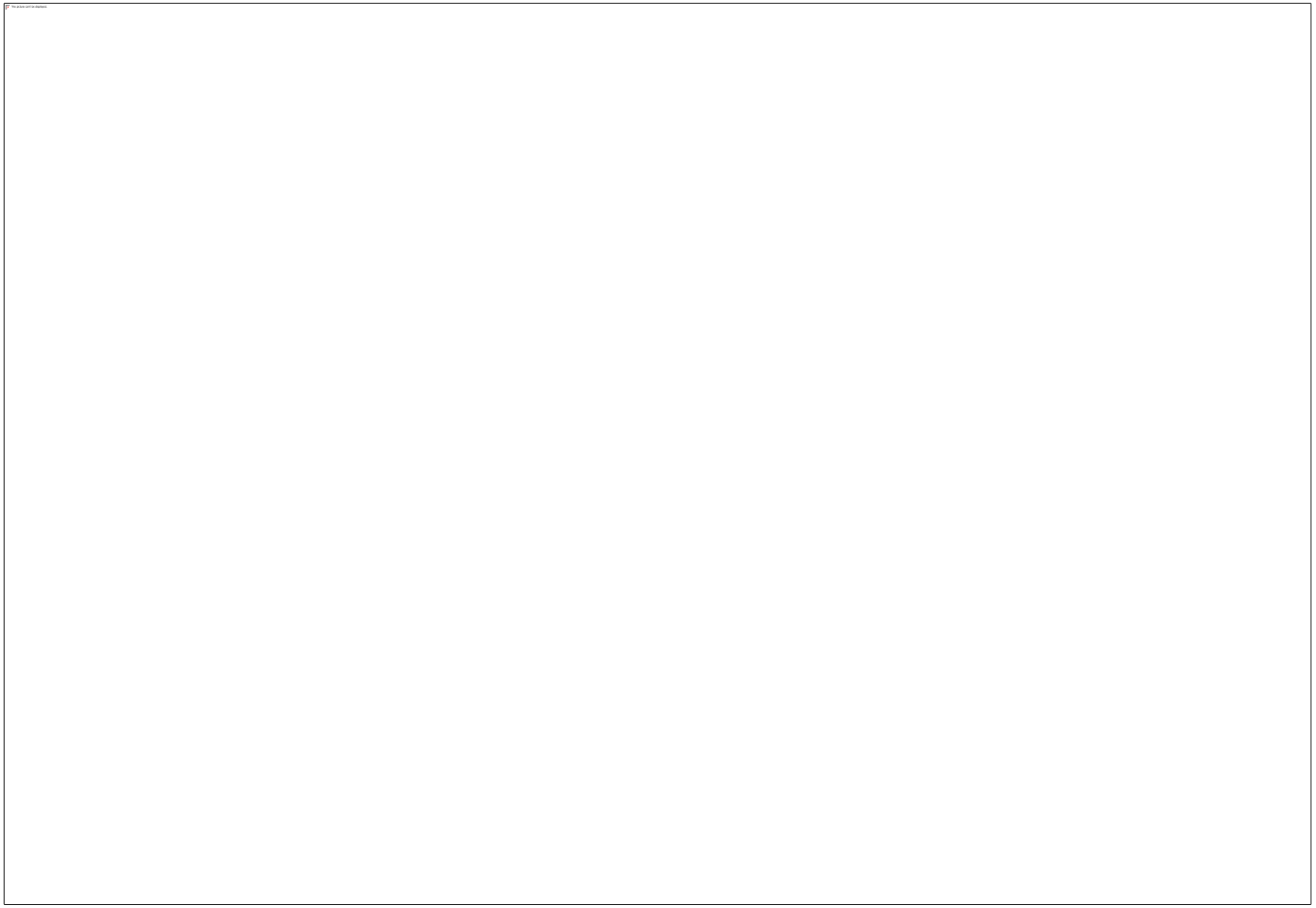
Comparison of internal sorting algorithms with sorted data



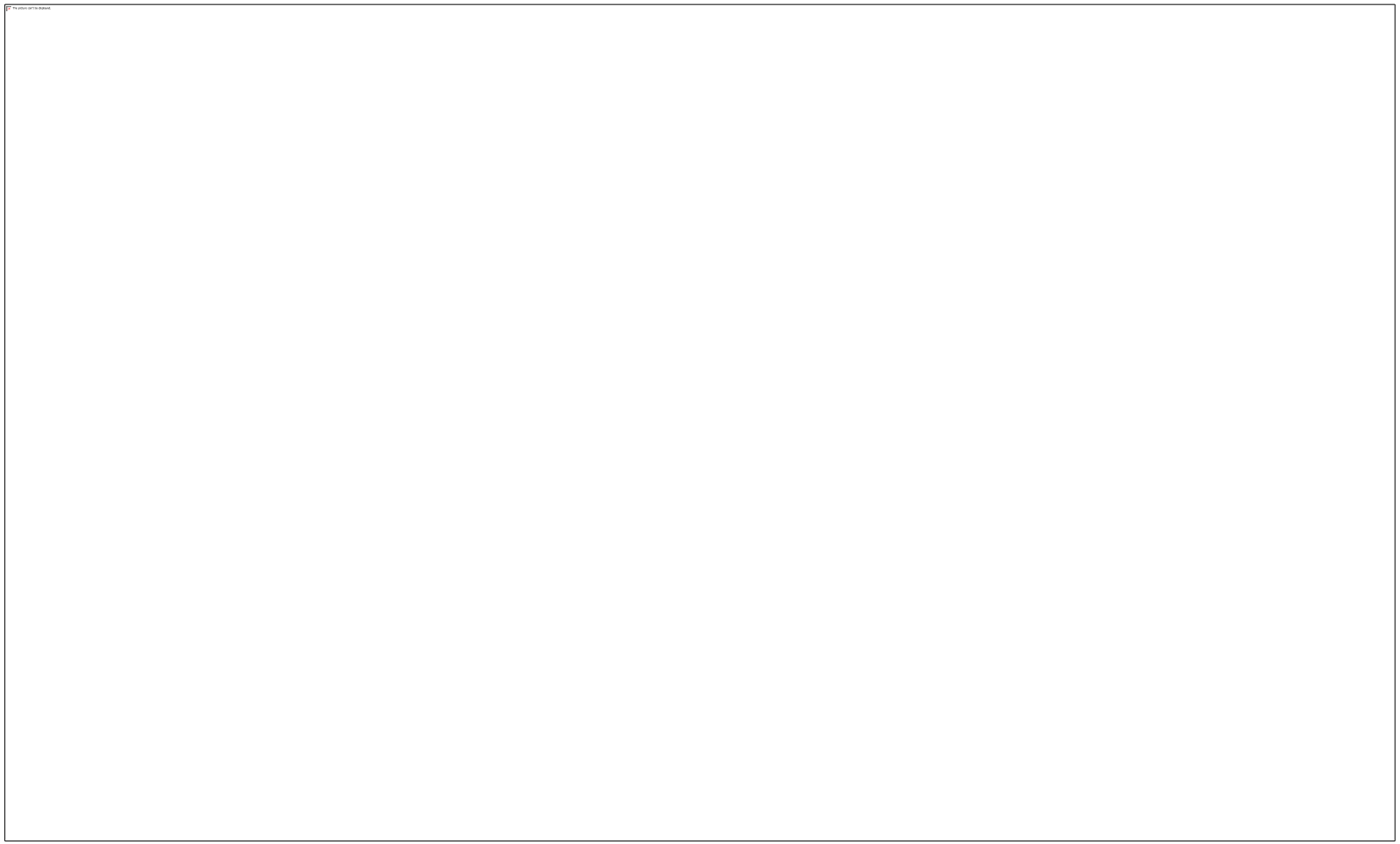
Comparison of internal sorting algorithms with repeated data



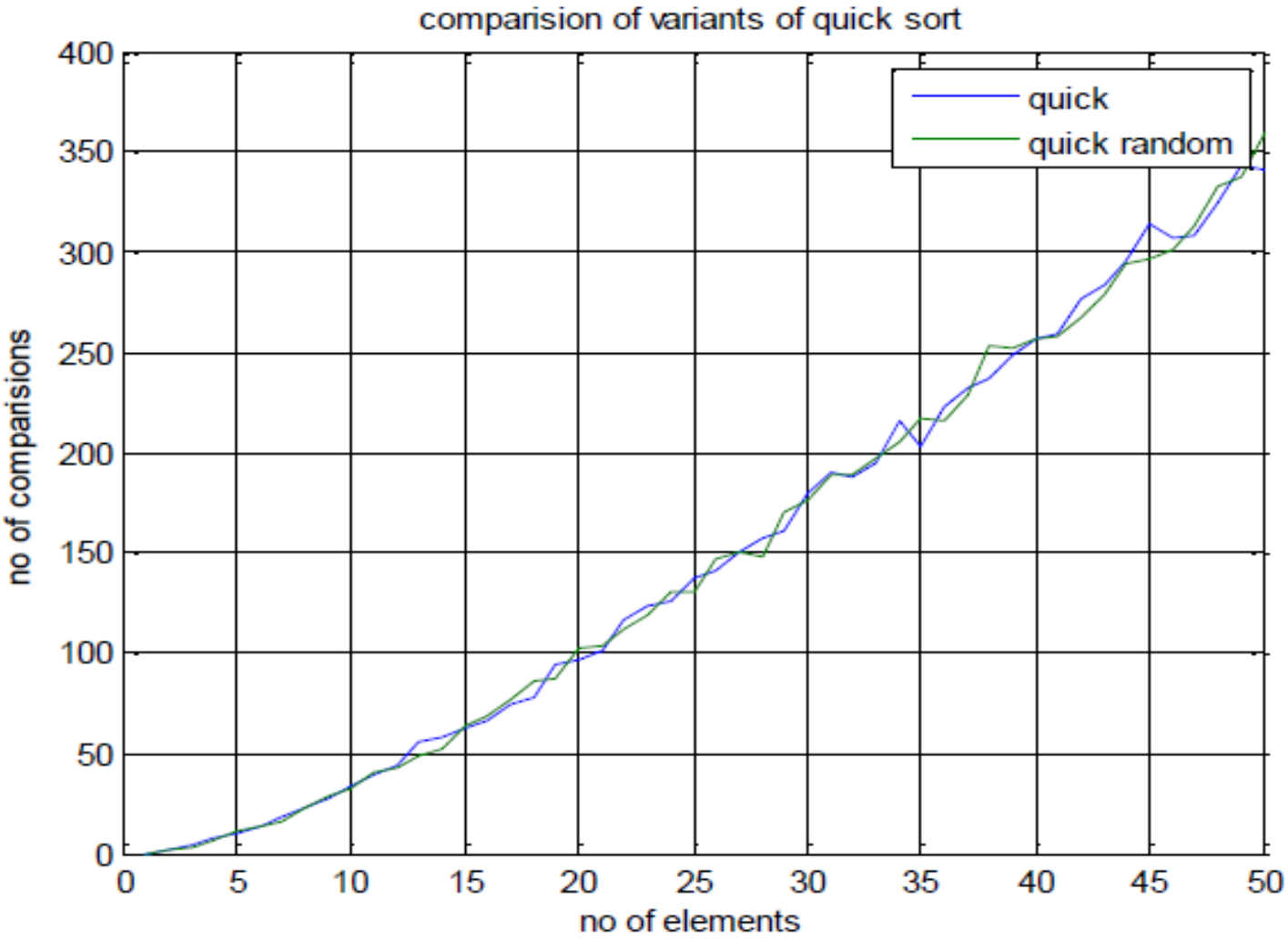
Comparison of internal sorting algorithms with reverse sorted data



Question



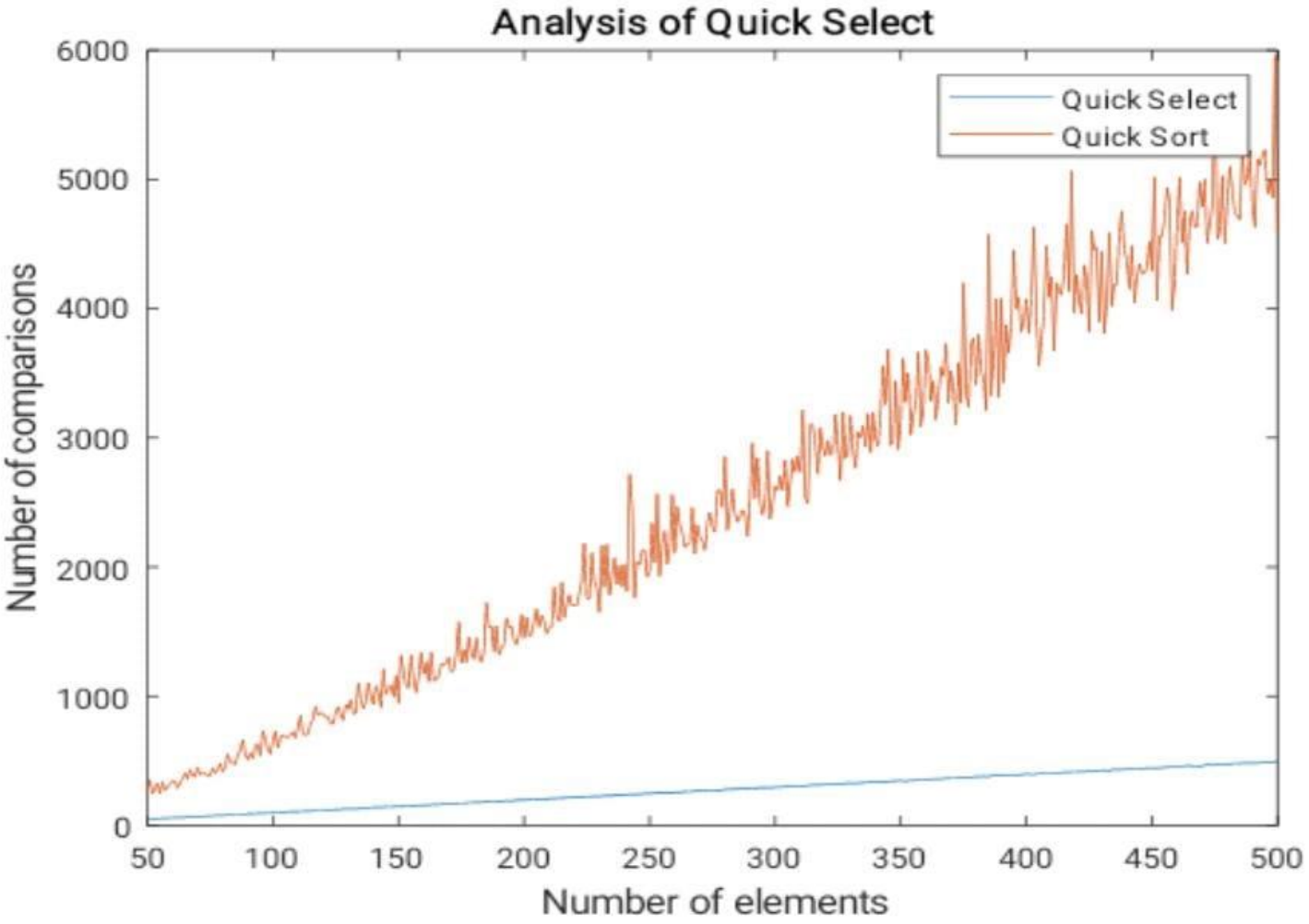
Variants of QUICK SORT



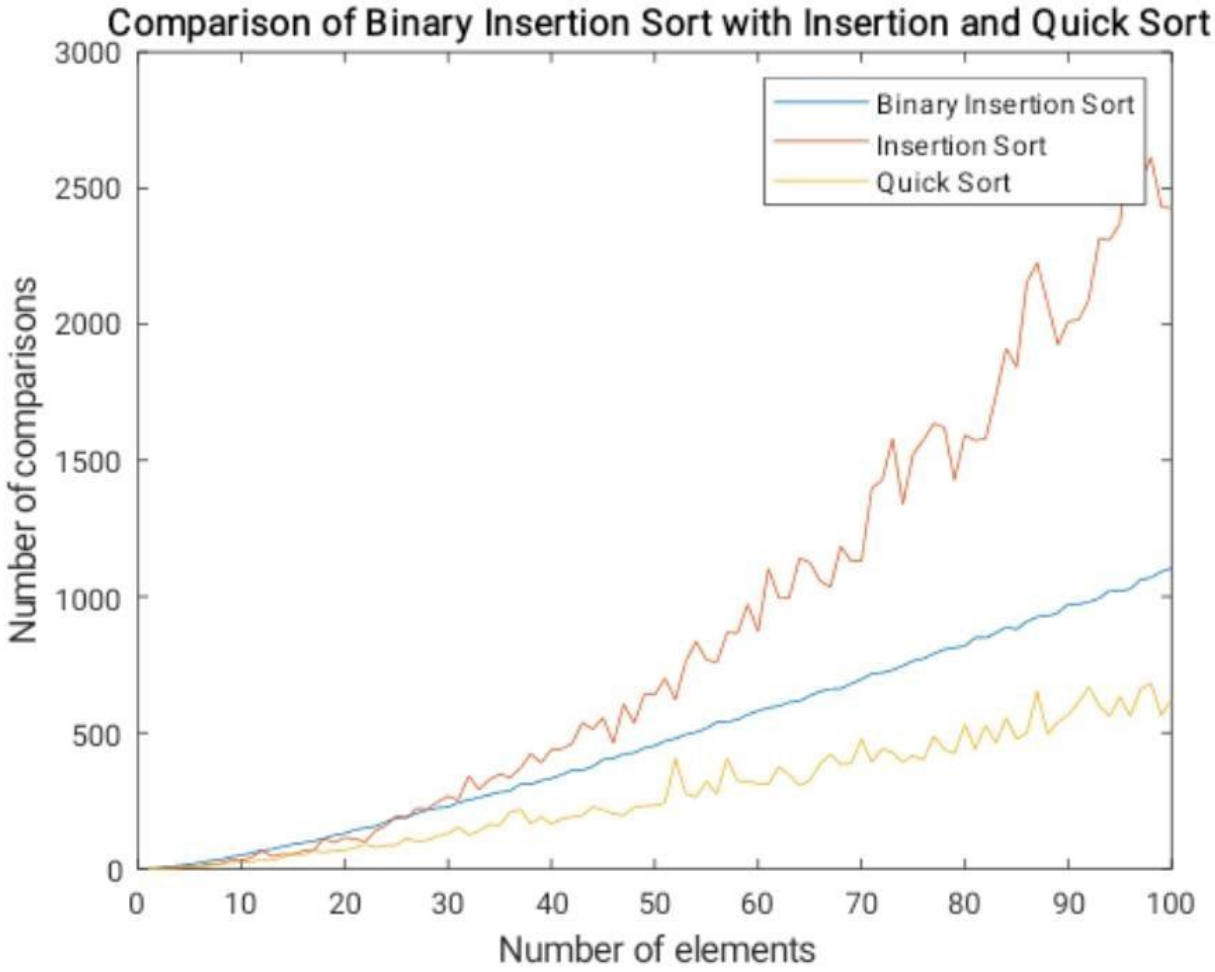
QUICK SELECT

- Use the **QUICK SELECT** algorithm to find **3rd largest element** in an array of n integers. Analyze the performance of **QUICK SELECT** algorithm for the different instance of size 50 to 500 element. Record your observation with the *number of comparison made* vs. *instance*.

QUICK SELECT



BINARY INSERTION SORT



BINARY INSERTION SORT

- **Binary Insertion sort** is a variant of Insertion sorting in which proper location to insert the selected element is found using the **binary search**. Compare the performance of Binary insertion sort with Quick sort and insertion sort. Run the experiment for at least 10 iterations for an instance and compare your results and comment on your simulation results.

Algorithm linear search

Algorithm 1.1 LINEARSEARCH

Input: An array $A[1..n]$ of n elements and an element x.

Output: j if $x = A[j], 1 \leq j \leq n$, and 0 otherwise.

1. $j \leftarrow 1$
2. **while** $(j < n)$ **and** $(x \neq A[j])$
3. $\quad j \leftarrow j + 1$
4. **end while**
5. **if** $x = A[j]$ **then return** j **else return** 0

Algorithm binary search

Algorithm 1.2 BINARYSEARCH

Input: An array $A[1..n]$ of n elements sorted in nondecreasing order and an element x.

Output: j if $x = A[j], 1 \leq j \leq n$, and 0 otherwise.

1. $low \leftarrow 1$; $high \leftarrow n$; $j \leftarrow 0$
2. **while** $(low \leq high)$ **and** $(j = 0)$
3. $\quad mid \leftarrow \lfloor (low + high)/2 \rfloor$
4. $\quad$ **if** $x = A[mid]$ **then** $j \leftarrow mid$
5. $\quad$ **else if** $x < A[mid]$ **then** $high \leftarrow mid - 1$
6. $\quad$ **else** $low \leftarrow mid + 1$
7. **end while**
8. **return** j

Merging two Sorted Lists

Algorithm 1.3 MERGE

Input: An array $A[1..m]$ of elements and three indices p, q and r, with $1 \leq p \leq q < r \leq m$, such that both the subarrays $A[p..q]$ and $A[q+1..r]$ are sorted individually in nondecreasing order.

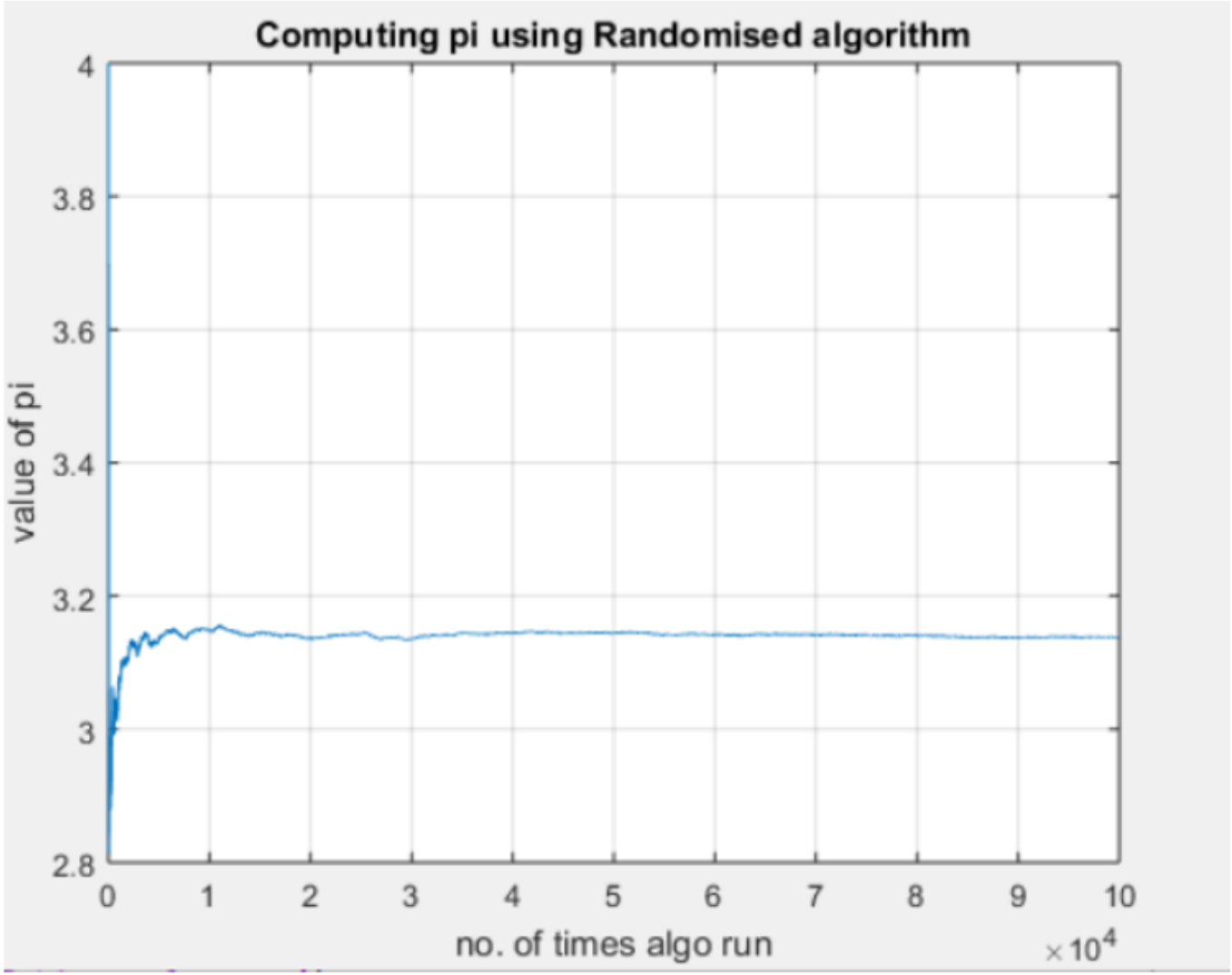
Output: $A[p..r]$ contains the result of merging the two subarrays $A[p..q]$ and $A[q+1..r]$.

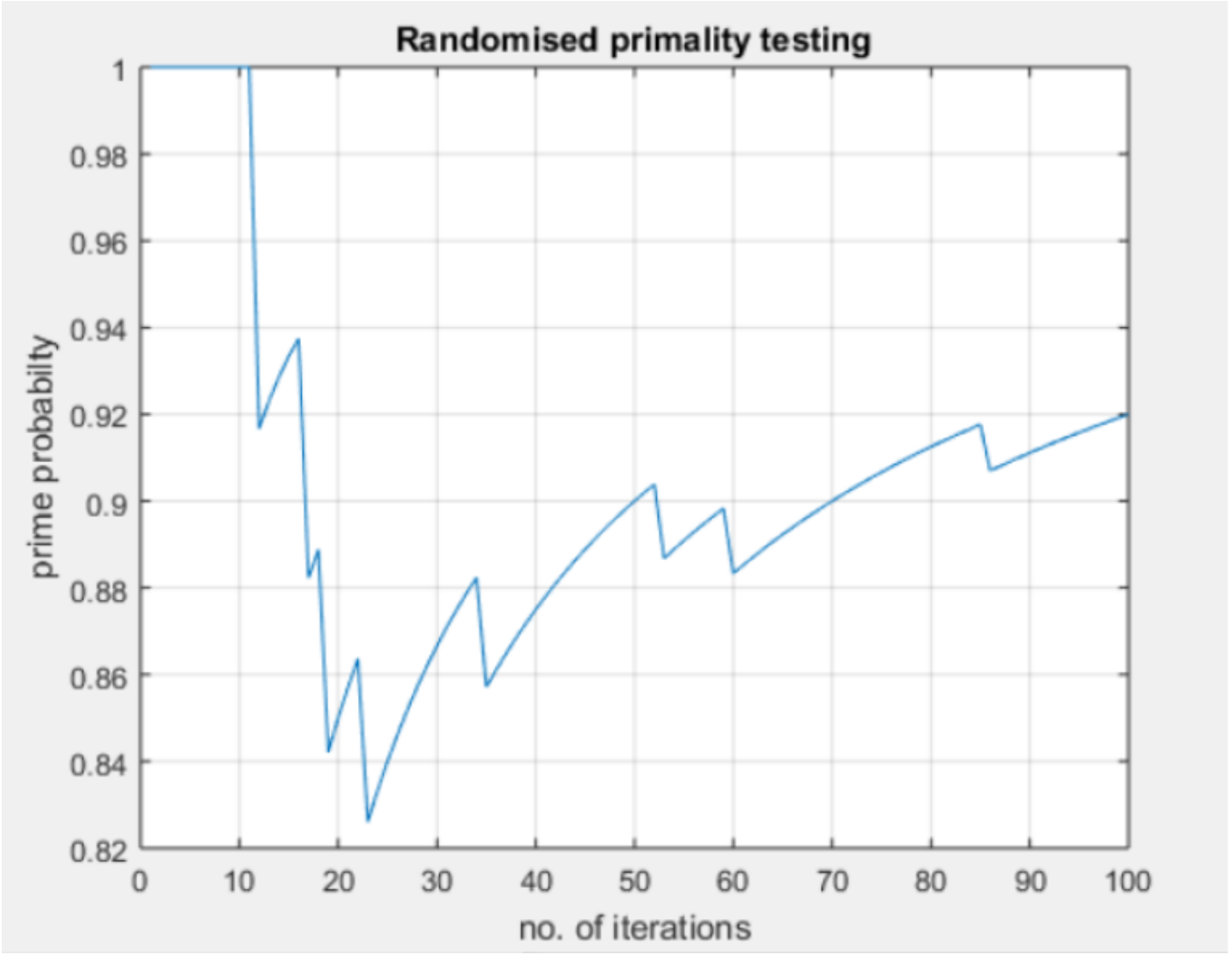
```
1.  comment: B[p..r] is an auxiliary array.
2.  s ← p;    t ← q + 1;    k ← p
3.  while s ≤ q and t ≤ r
4.      if A[s] ≤ A[t] then
5.          B[k] ← A[s]
6.          s ← s + 1
7.      else
8.          B[k] ← A[t]
9.          t ← t + 1
10.     end if
11.     k ← k + 1
12. end while
13. if s = q + 1 then B[k..r] ← A[t..r]
14. else B[k..r] ← A[s..q]
15. end if
16. A[p..r] ← B[p..r]
```

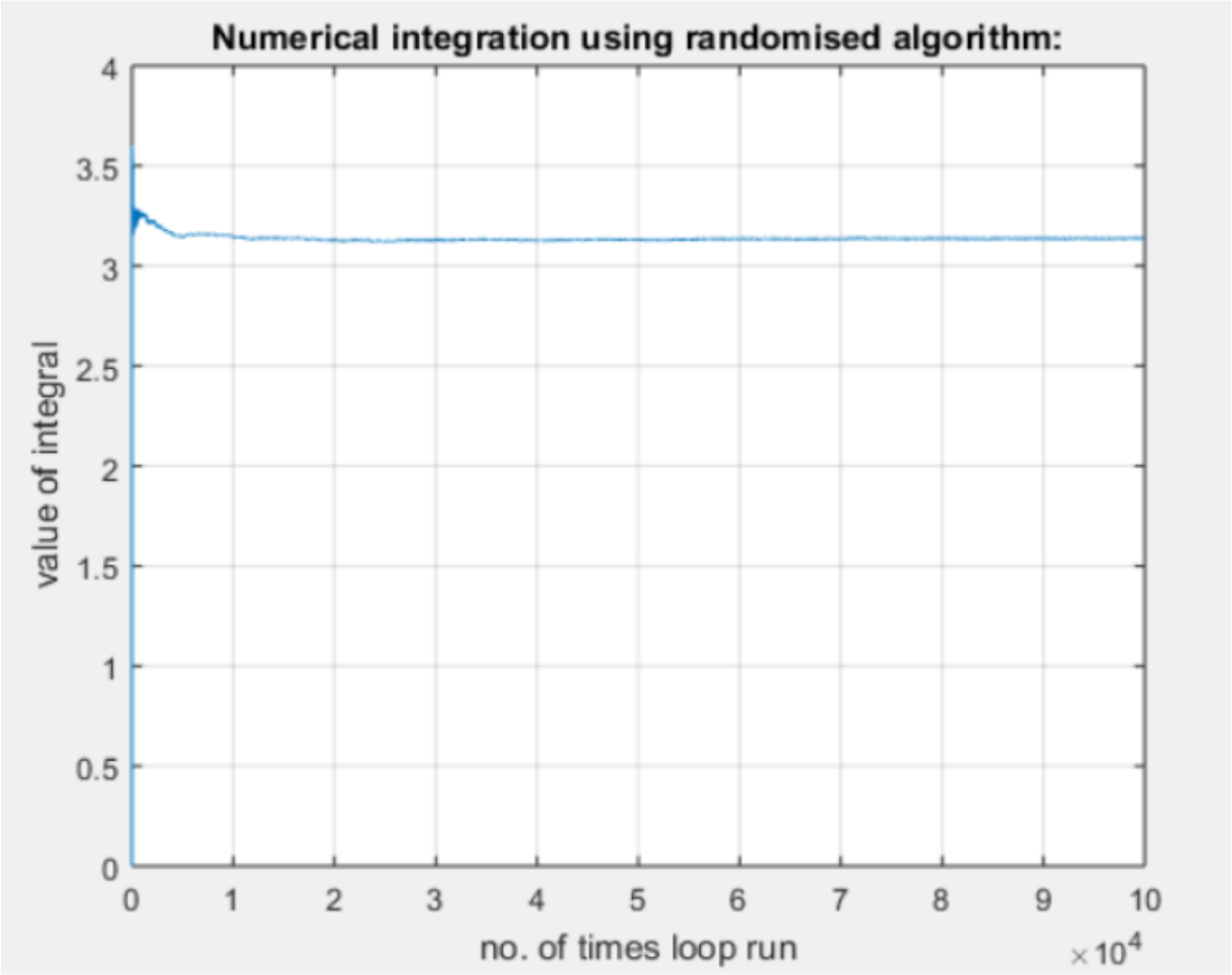
Limitations of Experiments

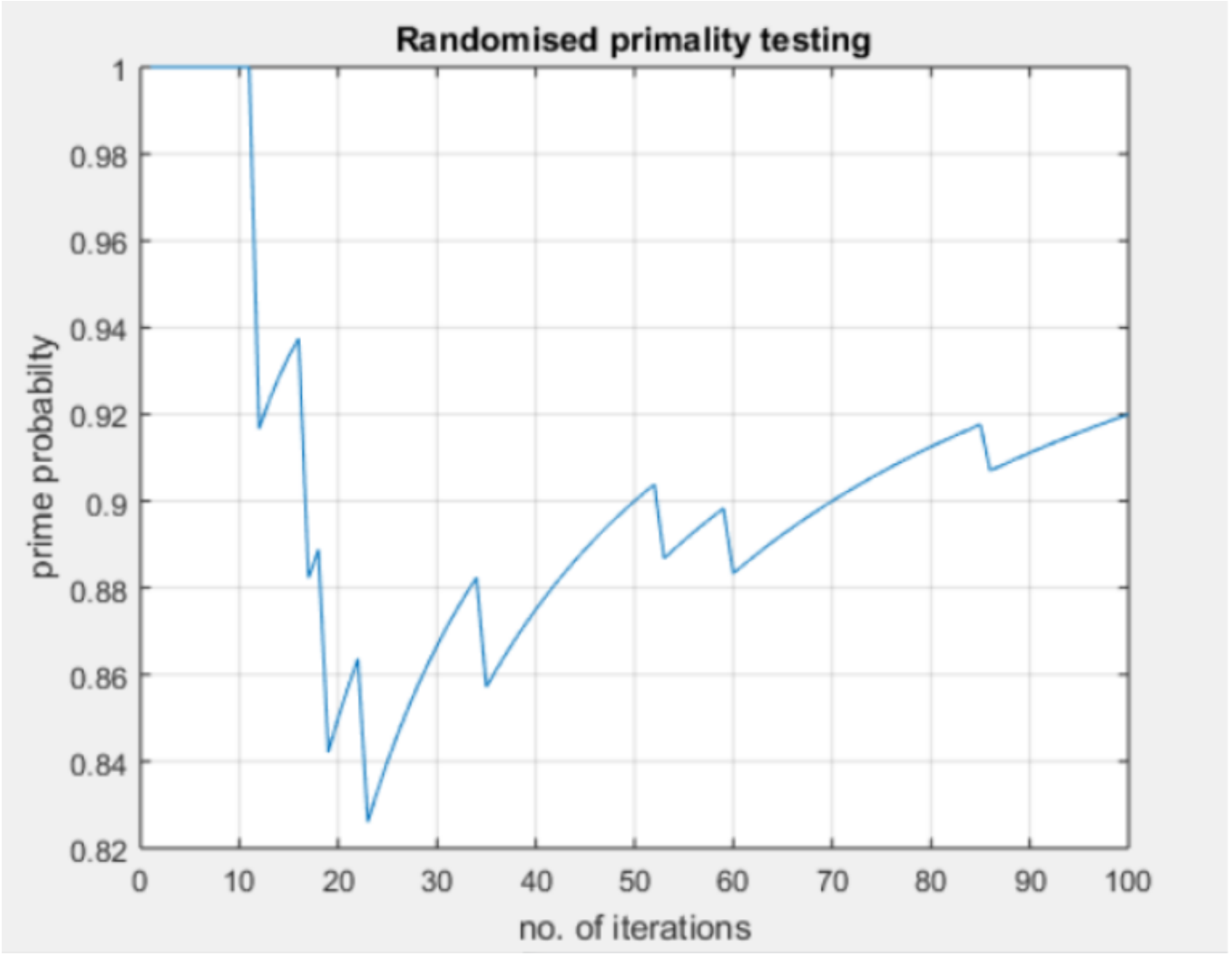
- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments must be used
- Experimental data though important is not sufficient

Compute Π using randomized algorithm




Randomised primality testing
prime probability
no. of iterations
1
0.98
0.96
0.94
0.92
0.9
0.88
0.86
0.84
0.82
0
10
20
30
40
50
60
70
80
90
100


Numerical integration using randomised algorithm:
value of integral
no. of times loop run
$\times 10^4$
0
0.5
1
1.5
2
2.5
3
3.5
4
0
1
2
3
4
5
6
7
8
9
10


Randomised primality testing
prime probability
no. of iterations
1
0.98
0.96
0.94
0.92
0.9
0.88
0.86
0.84
0.82
0
10
20
30
40
50
60
70
80
90
100

Validating a simulation result

How one can validate simulation results:

1. **Compare with Analytical Solutions:** If the system being simulated has a known analytical or theoretical solution, compare the simulation results against these benchmarks to check for consistency.
2. **Experimental Validation:** Compare simulation results with real-world experimental data. Ensure the conditions in the simulation closely match those of the real-world experiments.
3. **Sensitivity Analysis:** Vary key input parameters in the simulation and observe whether the results behave as expected. This checks the model's robustness and logical consistency.
4. **Statistical Validation:** Use statistical tests (e.g., t-test, chi-square test) to evaluate if the simulated results fall within the confidence interval of experimental or historical data.
5. **Visual Comparison:** Plot the simulation results against real or expected data. Look for trends, patterns, and overlaps (e.g., time series, histograms, scatter plots).

How one can validate simulation results:

6. **Peer Review and Expert Opinion**: Have domain experts review the simulation assumptions, parameters, and results to confirm that the outcomes are realistic and reliable.
7. **Cross-Model Comparison**: Compare your simulation results with those obtained from other validated models or simulations of similar systems.
8. **Test for Edge Cases**: Test the simulation under extreme conditions or boundary scenarios to ensure that it behaves realistically even at the limits.
9. **Verification of Input Data**: Validate the accuracy of input parameters and assumptions used in the simulation, as errors in input can lead to invalid results.
10. **Repeatability:** Re-run the simulation multiple times to ensure the results are consistent, especially for stochastic models.

ADVANCED DATA STRUCTURE AND PROBLEM SOLVING

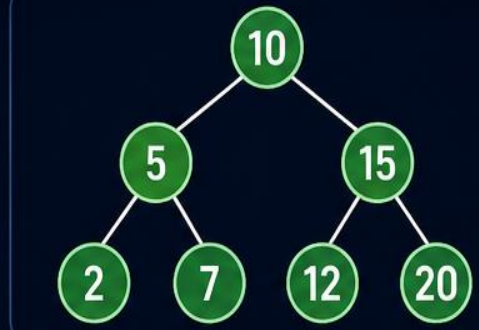
BUILDING EFFICIENT SOLUTIONS FOR COMPLEX PROBLEMS

- ✓ Advanced Data Structures
- ✓ Algorithm Design Techniques
- ✓ Complexity Analysis
- ✓ Optimization Strategies
- ✓ Real-world Problem Solving

</>

THINK
DESIGN
IMPLEMENT
OPTIMIZE

STRONGER STRUCTURES, SMARTER SOLUTIONS

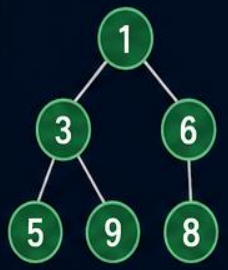


BINARY SEARCH TREE

GRAPH

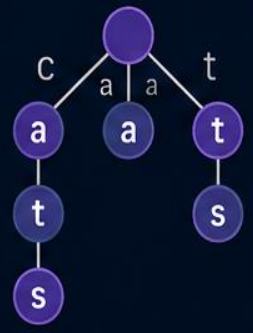
```
vector<int> dijkstra(int src) {
 priority_queue<pair<int,int>,
    vector<pair<int,int>>,
    greater<pair<int,int>>> pq;
 vector<int> dist(V, INF);
 dist[src] = 0;
 pq.push({0, src});
 ...
 return dist;
}
```

HEAP



HASH TABLE

TRIE



Suggested Design Assignments

#	Real-World Problem	Data Structure / Algorithm	Expected Outcome
1	**Hospital Emergency Management**	Priority Queue / Heap	Design a system that prioritizes patients based on severity, age, and waiting time.
2	**University Course Registration**	Hash Table + BST	Fast student/course lookup, registration, and conflict detection.
3	**Google Maps–like Route Planner**	Graph + Dijkstra / A*	Find shortest/fastest routes between locations.
4	**Social Network Friend Recommendation**	Graph + BFS/DFS	Recommend friends based on common connections.
5	**E-Commerce Product Search**	Trie + Hash Table + BST	Implement prefix search, product lookup, and price-based queries.
6	**Online Food Delivery System**	Heap + Graph + Hashing	Assign delivery orders to riders and optimize delivery routes.
7	**Airport Flight Management**	Graph + Priority Queue	Manage flights, connections, delays, and shortest routes.
8	**Library Management 2.0**	Hash Table + BST	Efficient book search, issue/return, and sorted catalog management.
9	**Real-Time Traffic Management**	Graph + Priority Queue	Identify congested routes and suggest alternative routes.
10	**File-System Simulator**	Tree + Hash Table	Design directory/file organization with search and navigation.

Suggested Design Assignments

#	Real-World Problem	Data Structure / Algorithm	Expected Outcome
11	**Web Search Engine Prototype**	Inverted Index + Hash Table + Heap	Search documents using keywords and rank results.
12	**CPU Process Scheduler**	Heap / Priority Queue	Implement priority, shortest-job, and deadline-based scheduling.
13	**Network Packet Routing**	Graph + Queue	Design efficient packet routing and congestion handling.
14	**Text Autocomplete System**	Trie	Build predictive text/autocomplete functionality.
15	**Plagiarism Detection System**	Hashing + String Algorithms	Detect similar passages efficiently.
16	**Online Shopping Cart & Recommendation**	Hash Table + Graph	Manage products and recommend related products.
17	**Disaster Evacuation Planner**	Graph + BFS/Dijkstra	Find safe and efficient evacuation routes.
18	**Campus Navigation System**	Graph + MST + Shortest Path	Model campus locations and optimize movement.
19	**Banking Transaction System**	Hash Table + BST	Manage accounts, transactions, and range-based queries.
20	**Smart Parking Management**	Hashing + Heap + Graph	Allocate parking spaces and optimize vehicle movement.

Five assignments recommend

Assignment 1 — Intelligent Hospital Emergency Management

- Design a patient-management system using **Priority Queue/Heap**. Students must develop a priority function considering severity, waiting time, and emergency status.

Assignment 2 — Intelligent Campus Route Planner

- Model the campus as a **weighted graph** and implement **Dijkstra/A*** for optimal routing. *Extension*: dynamically handle blocked roads.

Assignment 3 — Smart E-Commerce Search Engine

- Use **Trie + Hash Table + Heap** to implement keyword search, autocomplete, product lookup, and ranking.

Assignment 4 — Social Network Analysis

- Represent users and relationships using a **Graph**. Implement BFS/DFS, degree analysis, mutual-friend recommendation, and community identification.

Assignment 5 — Disaster Evacuation System

- Design a system that identifies **safe evacuation routes** using graphs and shortest-path algorithms. Extension: dynamically change edge weights when roads become unavailable.

RECOMMENDED ASSIGNMENT STRUCTURE

From Problem to Solution

01	**PROBLEM DEFINITION**	Clearly define the real-world problem. Identify the inputs, outputs and constraints.
02	**REAL-WORLD REQUIREMENTS**	List and analyze the requirements. Understand the stakeholders and use cases.
03	**DATA STRUCTURE SELECTION & JUSTIFICATION**	Choose the most suitable data structure(s). Justify your choice with reasons.
04	**ABSTRACT DATA TYPE / SYSTEM MODEL**	Design the ADT or system model. Define operations and their behavior.
05	**ALGORITHM DESIGN**	Develop the algorithm(s) to solve the problem. Use pseudocode / flowchart as needed.
06	**IMPLEMENTATION**	Implement the solution in the chosen programming language.
07	**COMPLEXITY ANALYSIS**	Analyze time and space complexity. Justify the efficiency of your solution.
08	**TEST CASES & PERFORMANCE EVALUATION**	Design test cases (normal, edge, stress). Evaluate correctness and performance.
09	**COMPARISON WITH ALTERNATIVE APPROACH**	Compare with another data structure or algorithm. Discuss trade-offs.
10	**CONCLUSIONS & POSSIBLE IMPROVEMENTS**	Summarize findings, limitations, and suggest future improvements.

GOALS

- Develop problem-solving skills
- Choose the right data structure
- Design efficient algorithms
- Analyze and evaluate solutions
- Apply to real-world scenarios

Multicore architecture
&
Programming

PROGRAMMING FOR INTEL MULTICORE AND PARALLEL PROCESSORS

Design. Parallelize. Optimize. Accelerate.

- **MULTICORE POWER** — Do more in parallel
- **HIGH PERFORMANCE** — Lower time, higher throughput
- **SCALABLE SOLUTIONS** — From desktops to data centers
- **NEXT-GEN PROGRAMMING** — OpenMP, oneTBB, oneAPI, SIMD

COURSE OUTCOMES

- Understand multicore architecture and memory hierarchy
- Design and implement parallel programs using OpenMP, C/C++ threads, and Intel oneTBB
- Apply SIMD and vectorization for performance
- Analyze, optimize and evaluate parallel programs
- Develop real-world parallel applications

COURSE MODULES

1 INTRODUCTION TO MULTICORE COMPUTING
- Evolution to multicore
- Parallelism vs Concurrency
- Shared-memory model

2 INTEL MULTICORE ARCHITECTURE
- Cores, threads, caches
- Cache hierarchy & coherence
- NUMA and memory system
- SIMD & vector units

3 PARALLEL PROGRAMMING FUNDAMENTALS
- Task, data & thread parallelism
- Work decomposition
- Synchronization & communication

4 INTEL THREADING TECHNOLOGIES
- Hyper-Threading Technology
- Intel Threading Building Blocks (oneTBB)
- OpenMP support
- Intel oneAPI ecosystem

5 OPENMP PROGRAMMING
- Parallel regions, for, sections
- Work sharing & scheduling
- Synchronization
- Critical, atomic, locks, reduction

6 C/C++ MULTITHREADING
- Threads and execution
- Mutexes and condition variables
- Race conditions & deadlocks

7 SIMD AND VECTOR PROGRAMMING
- Vectorization concepts
- Intel SIMD (SSE/AVX/AVX-512)
- Auto vs explicit vectorization
- Best practices

8 PARALLEL ALGORITHM DESIGN
- Parallel search & sort
- Matrix multiplication
- Reduction, prefix sum
- Graph algorithms

9 PERFORMANCE OPTIMIZATION
- Load balancing
- Cache & memory optimization
- False sharing & data locality
- Thread affinity & tuning

10 PERFORMANCE MEASUREMENT
- Speedup, efficiency, scalability
- Amdahl's & Gustafson's Law
- Profiling & benchmarking
- Tools: VTune, Perf, GProf

11 INTEL oneAPI PROGRAMMING MODEL
- DPC++ & SYCL concepts
- Single-source programming
- oneMKL, oneTBB integration
- Heterogeneous CPU/GPU

12 REAL-WORLD APPLICATIONS
- Image/Video processing
- Scientific computing
- Machine learning
- Big data, networks, simulation

LAB / PROGRAMMING ASSIGNMENTS

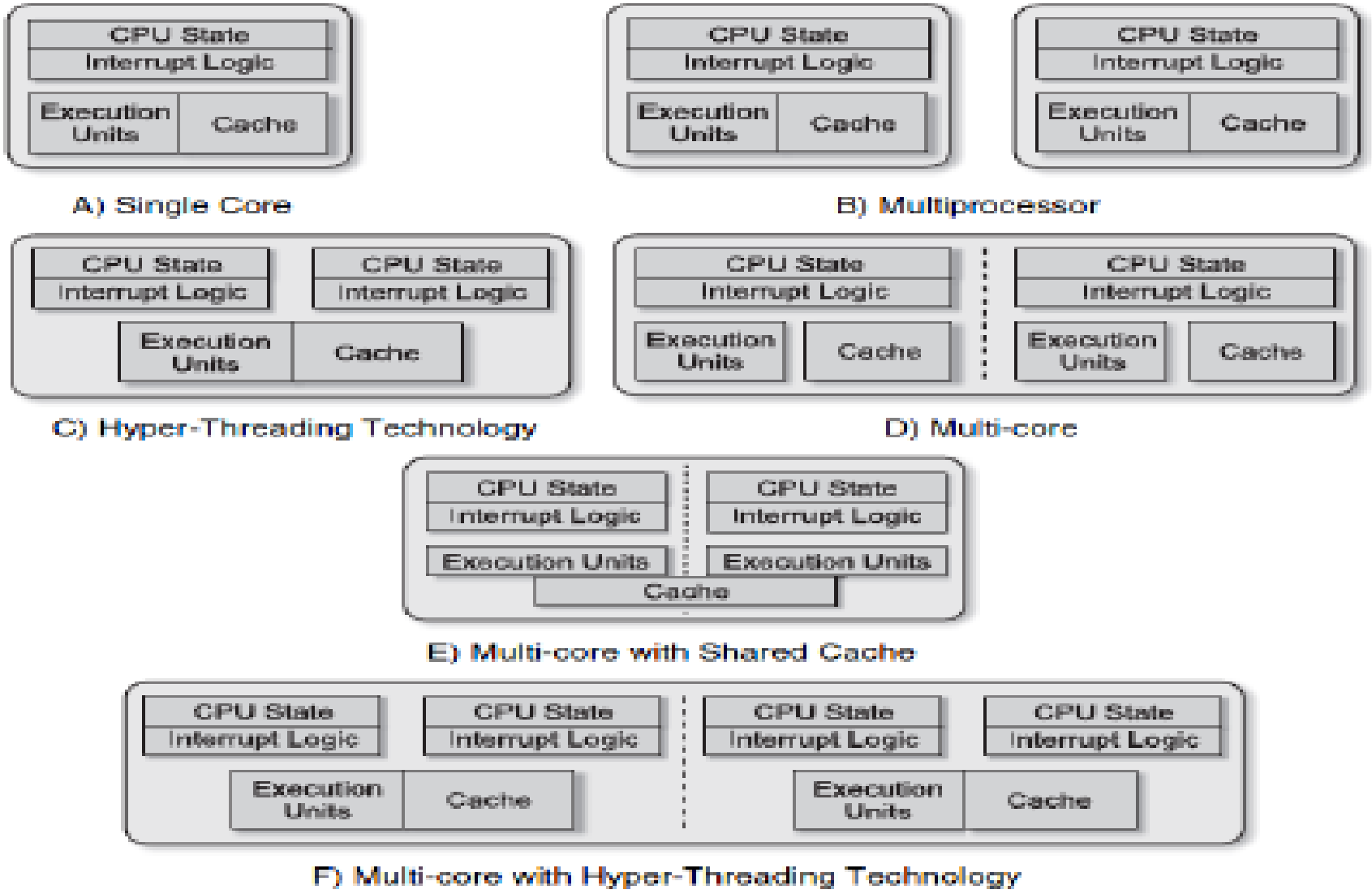
1. Sequential vs Multithreaded Matrix Multiplication
2. Parallel Array Processing using OpenMP
3. Parallel Merge Sort / Quick Sort
4. Parallel Reduction and Prefix Sum
5. Image Processing using Multithreading
6. Producer–Consumer using C++ Threads
7. Race Condition and Synchronization Demo
8. Cache Performance and False Sharing
9. SIMD Vectorization of Numerical Algorithms
10. Performance Comparison: 1, 2, 4, 8, ... Threads
11. Parallel Graph Traversal (BFS)
12. Intel oneAPI / DPC++ Mini Project

TOOLS & TECHNOLOGIES

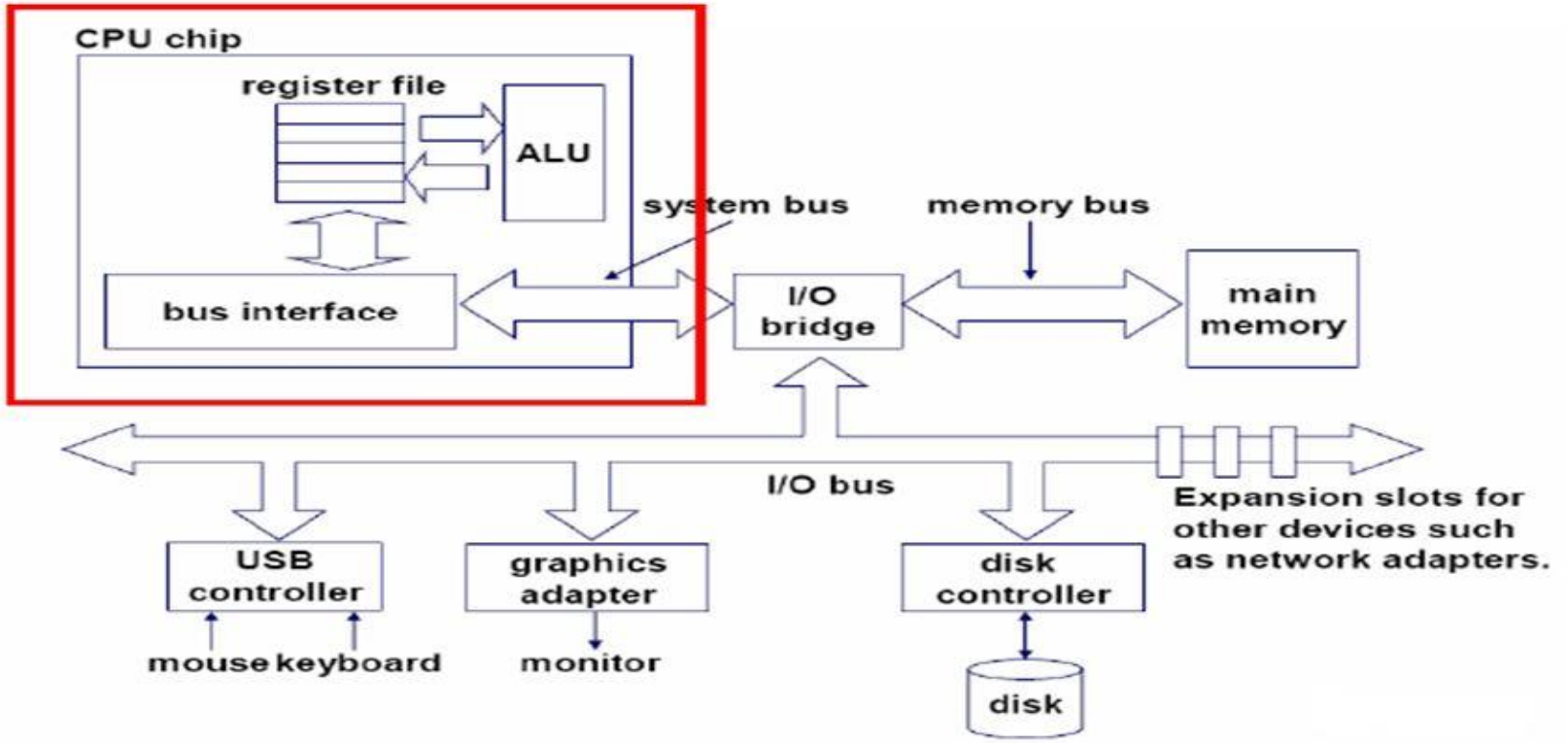
OpenMP · oneTBB · C++ Threads · oneAPI · Intel VTune Profiler

WHY THIS COURSE?

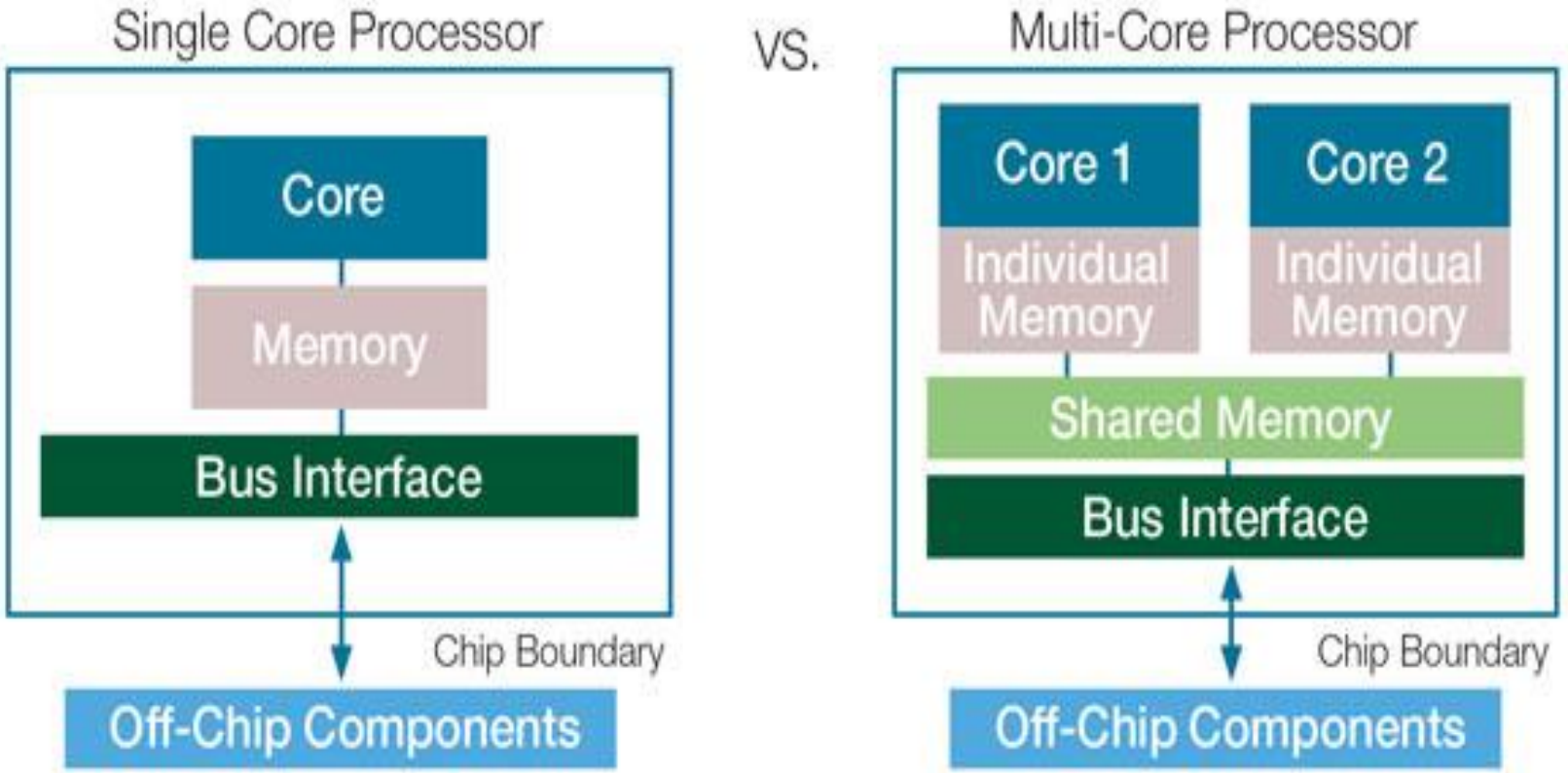
- Harness the power of modern multicore CPUs
- Build fast, scalable and efficient software
- Gain industry-relevant parallel programming skills
- Solve real-world problems with high performance


CPU State
Interrupt Logic
Execution Units
Cache
A) Single Core
B) Multiprocessor
C) Hyper-Threading Technology
D) Multi-core
E) Multi-core with Shared Cache
F) Multi-core with Hyper-Threading Technology

Single-core computer



Single Core Processor vs Multi Core Processor



Difference Between Multicore and Multiprocessor

- The **main difference** between multicore and multiprocessor is that the **multicore refers to a single CPU with multiple execution units while the multiprocessor refers to a system that has two or more CPUs.**
- Multicores have multiple cores or processing units in a single CPU. A multiprocessor contains multiple CPUs. Both multicore and multiprocessors help to speed up the computing process. A multicore does not require complex configurations like a multiprocessor. On the other hand, a multiprocessor is more reliable and capable of executing multiple programs. In brief, a multicore has a single CPU whereas a multiprocessor has many CPUs.

Single Core Processor vs Multi Core Processor

Parameter	Single-Core Processor	Multi-Core Processor
Number of cores on a die	Single	Multiple
Instruction Execution	Can execute Single instruction at a time	Can execute multiple instructions by using multiple cores
Gain	Speed up every program or software being executed	Speed up the programs which are designed for multi-core processors
Performance	Dependent on the clock frequency of the core	Dependent on the frequency, number of cores and program to be executed
Examples	Processor launched before 2005 like 80386,486, AMD 29000, AMD K6, Pentium I,II,III etc.	Processor launched after 2005 like Core-2-Duo,Athlon 64 X2, I3,I5 and I7 etc.

MULTICORE VERSUS MULTIPROCESSOR

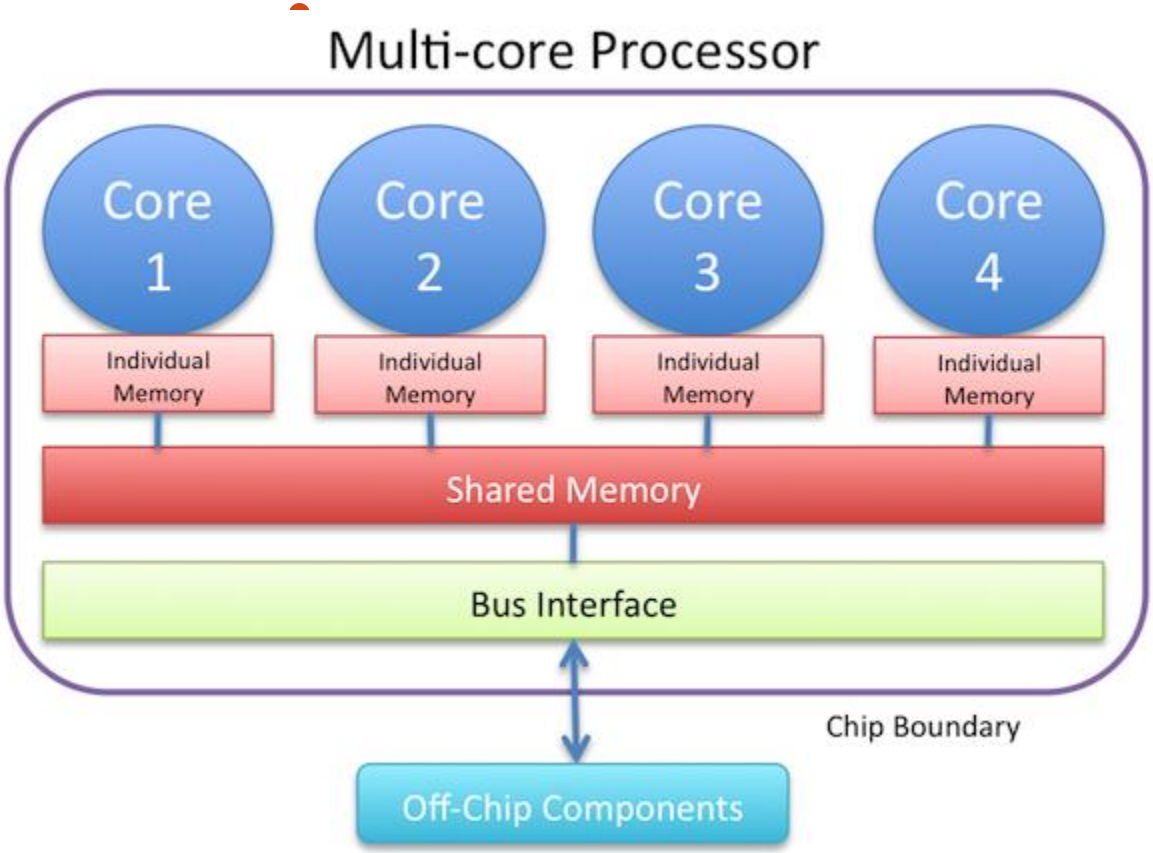
MULTICORE	MULTIPROCESSOR
A single CPU or processor with two or more independent processing units called cores that are capable of reading and executing program instructions	A system with two or more CPUs that allows simultaneous processing of programs
Executes a single program faster	Executes multiple programs faster
Not as reliable as a multiprocessor	More reliable since failure in one CPU will not affect the other
Have less traffic	Have more traffic

Visit www.PEDIAA.com

Suggested reading for Multicore Architecture

Mmulti-core architecture

- A **multi-core architecture** (or a chip multiprocessor) is a. general-purpose processor that consists of multiple. cores on the same die and can execute programs. simultaneously.



Multicore Architecture

- A multicore chip is a chip that has two or more processors. This processor configuration is referred to as CMP. CMPs currently range from dual core to octa - core.
- Hybrid multicore processors can contain different types of processors. The Cell broadband engine is a good example of a hybrid multicore.
- Multicore development can be approached from the bottom up or top down, depending on whether the developers in question are system programmers, kernel programmers, library developers, server developers, or application developers. Each group is faced with similar problems but looks at the cores from a different vantage point.

Multicore Architecture

- All developers that plan to write software that takes advantage of multiprocessor configurations should be familiar with the basic processor architecture of the target platform.
- The primary interface to the specific features of a multicore processor is the C/C++ compiler. To get the most from the target processor or family of target processors, the developer should be familiar with the options of the compiler, the assembler subcomponent of the compiler, and the linker.
- The secondary interface comprises the operating system calls and operating system synchronization and communication components.
- Parallel programming is the art and science of implementing an algorithm, a computer program, or a computer application using sets of instructions or tasks designed to be executed concurrently.
- Multicore application development and design is all about using parallel programming techniques and tools to develop software that can take advantage of CMP architectures.

INTEL COMPILER

High-Performance Compilers for Modern Intel® Architectures

Delivering maximum performance, scalability and productivity for C, C++, Fortran and more.

WHAT IS INTEL COMPILER?

Intel® oneAPI DPC++/C++ Compiler, Intel® C++ Compiler, Intel® Fortran Compiler and Intel® AI Analytics Toolkit compilers help developers build fast, efficient and reliable applications for Intel® CPUs, GPUs and accelerators.

Higher Performance
Generate highly optimized code for Intel microarchitectures.

Productive
Advanced optimizations with easy-to-use features and diagnostics.

Standards Compliant
Support for the latest C, C++ and Fortran standards.

Cross-Architecture
Build once and run on a wide range of Intel® platforms.

COMPILER WORKFLOW

Source Code (C / C++ / Fortran) → Intel Compiler (Front End) → Optimization (Analysis & Transformations) → Code Generation (Back End) → 1010 0101 … Optimized Binary

KEY OPTIMIZATION FEATURES

Auto Vectorization
Automatically uses SIMD instructions (SSE, AVX, AVX2, AVX-512) for maximum data parallelism.

Advanced Loop Optimizations
Loop unrolling, fusion, tiling and blocking for better cache usage and throughput.

Threading Support
Built-in support for OpenMP*, TBB and C++ threads for multi-core performance.

Memory Optimizations
Improved cache locality, prefetching and memory access patterns.

Interprocedural Optimization
Analyzes across functions and files to inline, eliminate redundancy and improve efficiency.

Profile-Guided Optimization
Uses runtime profile data to optimize critical code paths.

SUPPORTED LANGUAGES

C	C	High-performance C compilation
C++	C++	Modern C++ support (C++17/20/23)
F	Fortran	High-performance Fortran compilation
DPC++	SYCL/DPC++	Single-source programming for CPUs, GPUs and accelerators

TARGET ARCHITECTURES

Intel® Core™ Processors · Intel® Xeon® Processors · Intel® Arc™ GPUs · Intel® Gaudi® AI Accelerators

HOW TO USE

```
# Compile C / C++ code
icx -O3 -xHost -qopenmp -o app app.cpp

# Compile Fortran code
ifx -O3 -xHost -qopenmp -o app app.f90

# Generate vectorization report
icx -O3 -qopt-report=5 -o app app.cpp
```

INTEGRATED WITH INTEL® ONEAPI TOOLKIT

oneAPI

Seamlessly integrated with oneAPI libraries and tools end-to-end development.

oneMKL · oneDNN · oneTBB · oneData Analytics · oneAPI DPC++ Library

BENEFITS

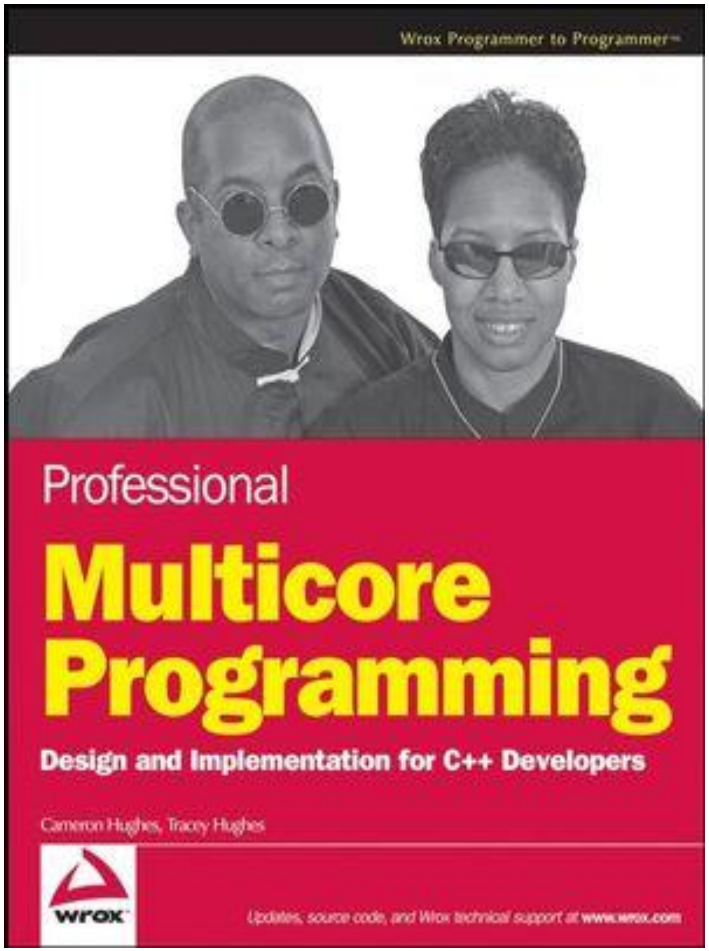
- Up to 2–5X performance improvement on Intel® architectures*
- Better scalability on multi-core and many-core systems
- Industry-proven reliability and enterprise-grade support
- Accelerate time-to-market with high performance code

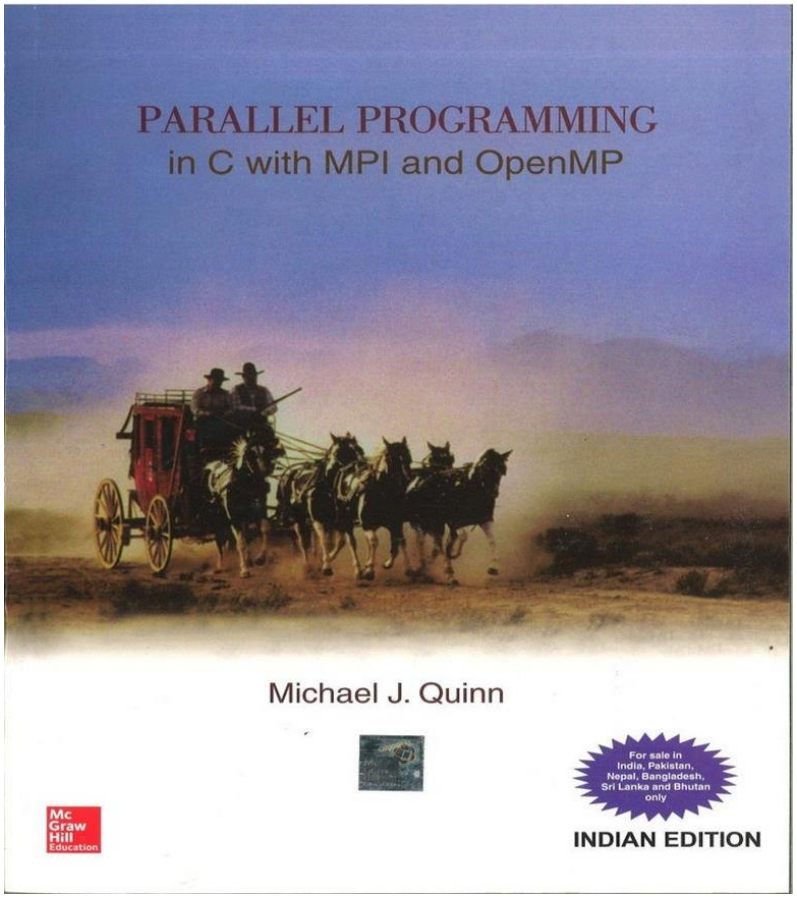
Learn more: intel.com/compilers

*Performance varies by workload, platform and configuration.

intel

Multicore Programming reference book





Intel Compiler

High Performance. Smarter Code. Faster Execution.

- Optimize Performance
- Improve Productivity
- Leverage Modern CPUs
- Scalable from Desktop to Cloud
- Cross-Platform & Future-Ready

What is Intel Compiler?

Intel® Compilers are a family of high-performance compilers that generate optimized code for Intel® architectures.

- ✔ For C, C++, Fortran & other languages
- ✔ Exploit multicore, SIMD, and vector instructions
- ✔ Deliver high performance, portability, and reliability

How Intel Compiler Works

Source Code → C / C++ / Fortran

Intel Compiler Front End → Parsing, Semantic Analysis, Optimization

Optimization Engine → Auto Vectorization, Loop Optimization, Parallelization

Target Code Generation → Intel® CPU Instructions

Optimized Executable → High Performance Execution

Why Choose Intel Compiler?

- ✔ Maximum performance on Intel CPUs
- ✔ Advanced optimizations (O3, IPO, IPA)
- ✔ Auto-vectorization with AVX/AVX-512
- ✔ Excellent support for OpenMP & MPI
- ✔ Integrated with Intel® oneAPI ecosystem
- ✔ Better performance with less code

Key Optimization Features

- **SIMD & Vectorization** — Uses AVX, AVX2, AVX-512 for data-level parallelism
- **Auto Parallelization** — Detects and parallelizes loops automatically
- **Memory Optimization** — Better cache usage, prefetching & locality
- **Whole Program Optimization** — Interprocedural analysis, link-time optimization

Example: Vectorization with Intel Compiler (C++)

```
01  #include <immintrin.h>
02  #include <stdio.h>
03
04  int main() {
05    float a[1000], b[1000], c[1000];
06    for (int i = 0; i < 1000; i++) {
07      c[i] = a[i] + b[i];
08    }
09    return 0;
10  }
```

Intel Compiler Optimizes

Generated SIMD Instructions (Example)

```
vmovaps   ymm0, YMMWORD PTR [rdi]
vmovaps   ymm1, YMMWORD PTR [rsi]
vaddps    ymm2, ymm0, ymm1
vmovaps   YMMWORD PTR [rdx], ymm2
```

- ✔ Uses AVX Instructions
- ✔ Processes 8 or 16 floats at a time

Intel® oneAPI Ecosystem

Supported Languages

C · C++ · Fortran · OpenMP · MPI · SYCL

Compilation Example (Linux / Windows)

```
# Using icx (Intel C++ Compiler)
icx -O3 -xHost -qopenmp -o myprog myprog.cpp

# Using ifx (Intel Fortran Compiler)
ifx -O3 -march=native -o myprog myprog.f90
```

Applications

- Scientific Computing
- AI / Machine Learning
- Image & Video Processing
- Engineering Simulation
- Data Analytics & Big Data
- High Performance Applications

Intel Compiler = Better Code + Better Performance + Better Productivity

Reference

[1] Sutter, Herb. "The Free Lunch Is Over: A Fundamental TurnTowardConcurrency in Software." Dr. Dobb's Journal, 30(3), March2005.

[2] Mattson, Timothy G.,Beverly A. Sanders, and Berna L. Massingill,Patternsfor Parallel Programming. Boston: Addison-Wesley, 2005.

[3] Moreabout multicores and multiprocessors

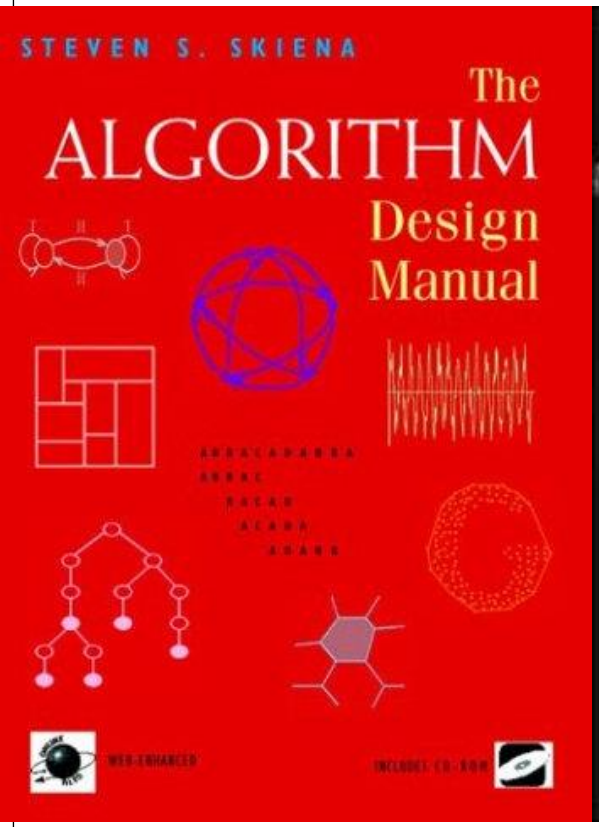
Sample problems

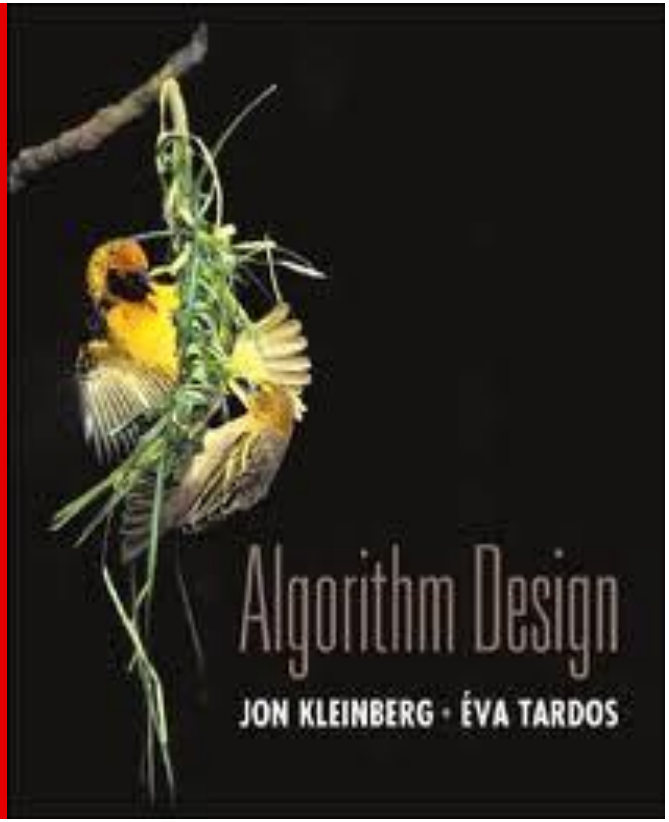
1. Write the most efficient algorithm you can think of for the following: Find the k-th item in an n-node doubly-linked list. What is the running time in terms of big-theta?
2. Write the most efficient algorithm you can think of for the following: Given a set of p points, find the pair closest to each other. Be sure and try a divide-and-conquer approach, as well as others. What is the running time in terms of big-theta?
3. Design and implement an algorithm that finds all the duplicates in a random sequence of integers.
4. Reconsider the three algorithms you just designed given the following change: the input has increased by 1,000,000,000 times.

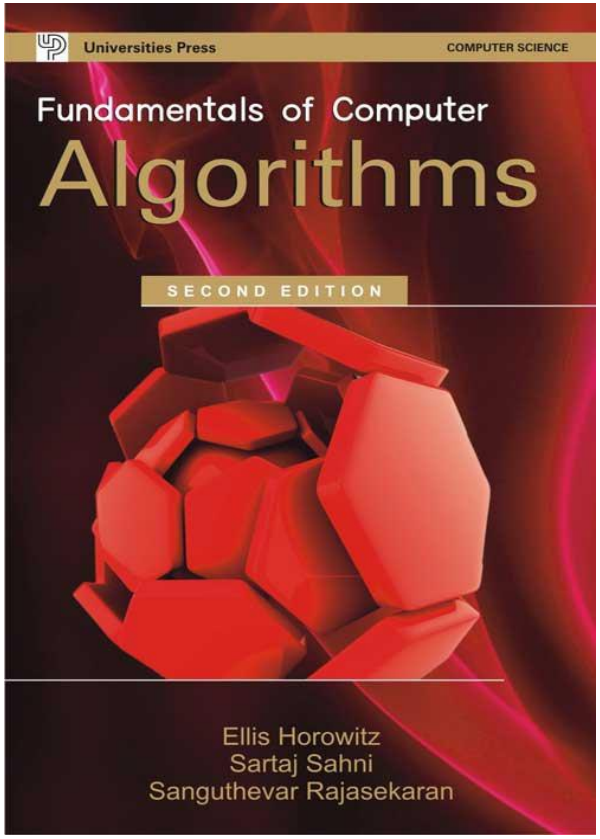
Sample problems

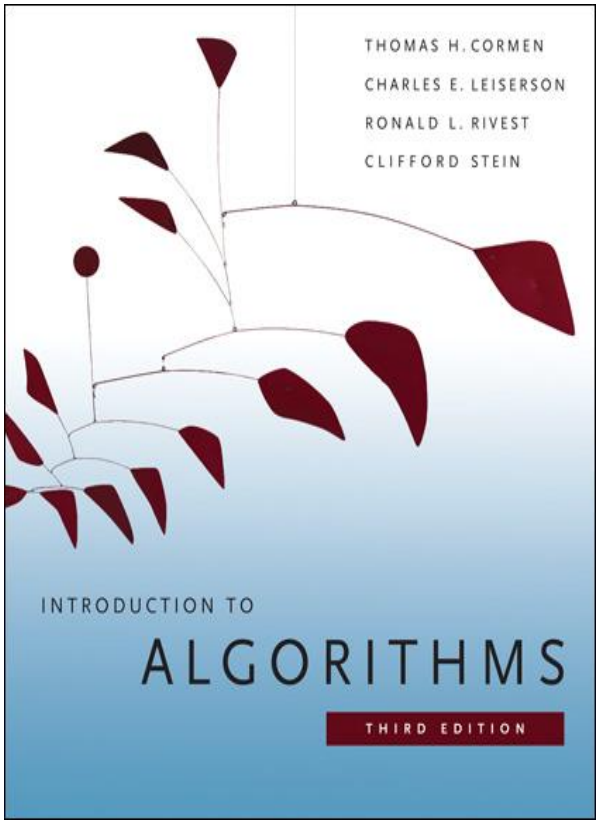
5. You have two arrays of integers. Within each array, there are no duplicate values but there may be values in common with the other array. Assume the arrays represent two different sets. Define an O(n log n) algorithm that outputs a third array representing the union of the two sets. The value "n" in O(n log n) is the sum of the sizes of the two input arrays.
6. You are given an increasing sequence of numbers u_1, u_2, … u_m, and a decreasing series of numbers d_1, d_2, …d_n. You are given one more number C and asked to determine if C can be written as the sum of one u_i and one d_j. There is an obvious brute force method, just comparing all the sums to C, but there is a much more efficient solution. Define an algorithm that works in linear time.
7. Design and implement a dynamic programming algorithm to solve the change counting problem. Your algorithm should always find the optimal solution--even when the greedy algorithm fails.

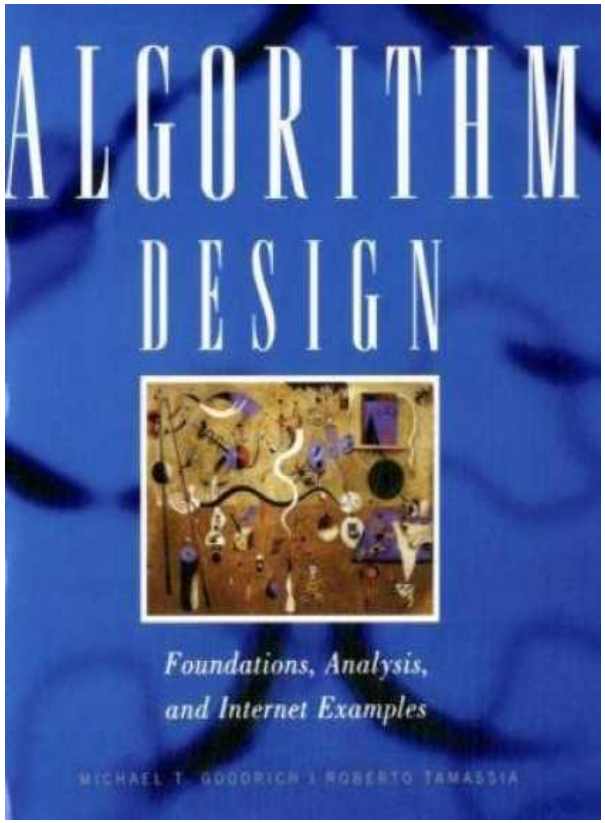
Reference:

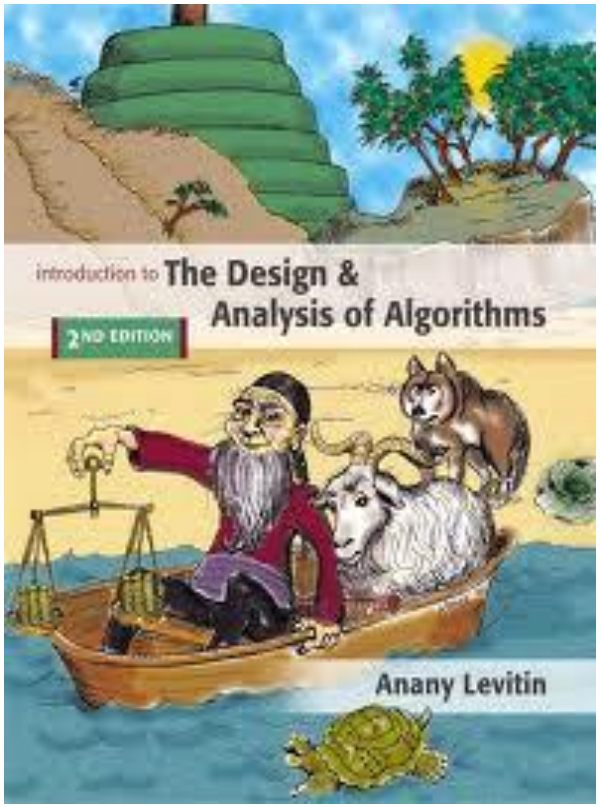

STEVEN S. SKIENA
The
ALGORITHM
Design
Manual

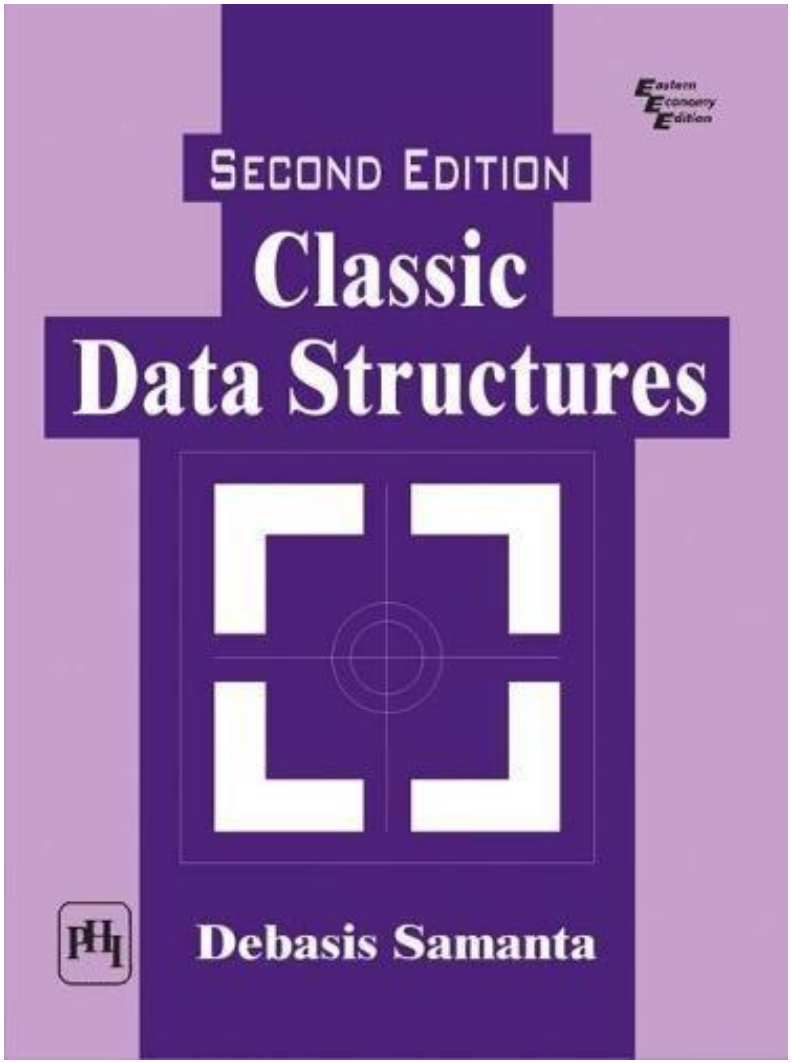

Algorithm Design
JON KLEINBERG · ÉVA TARDOS

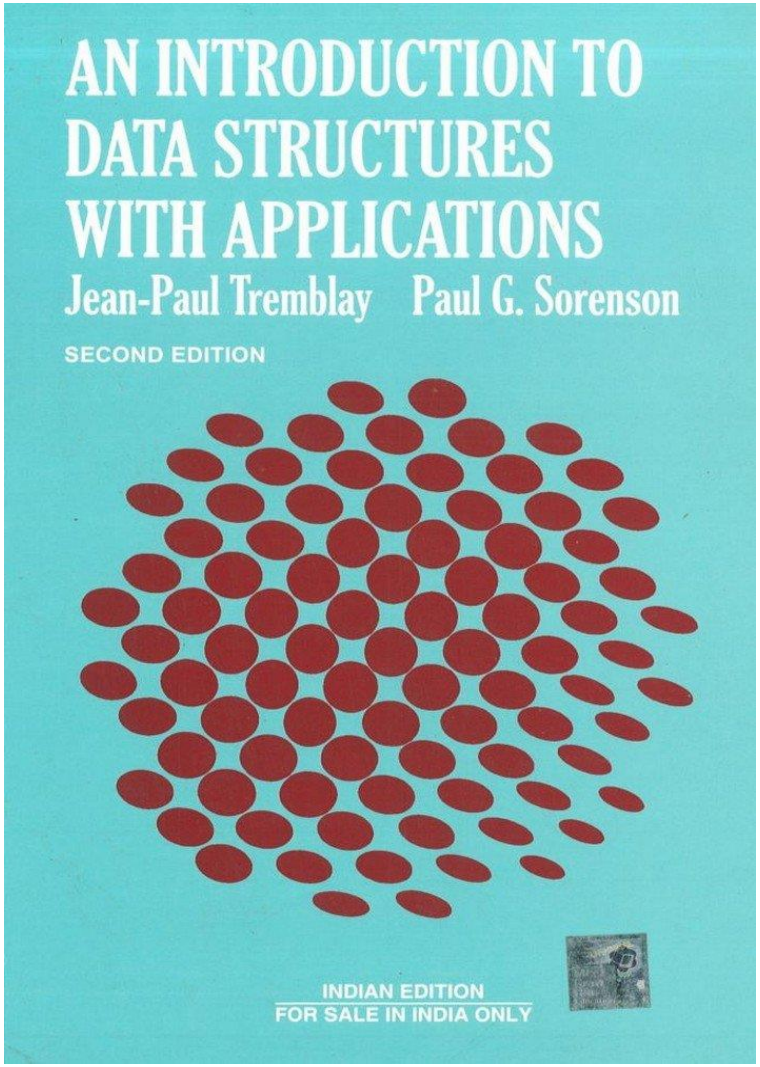

Universities Press
COMPUTER SCIENCE
Fundamentals of Computer
Algorithms
SECOND EDITION
Ellis Horowitz
Sartaj Sahni
Sanguthevar Rajasekaran


THOMAS H. CORMEN
CHARLES E. LEISERSON
RONALD L. RIVEST
CLIFFORD STEIN
INTRODUCTION TO
ALGORITHMS
THIRD EDITION


ALGORITHM
DESIGN
Foundations, Analysis,
and Internet Examples


Introduction to The Design &
Analysis of Algorithms
2ND EDITION
Anany Levitin


Eastern Economy Edition
SECOND EDITION
Classic
Data Structures
PHI
Debasis Samanta


AN INTRODUCTION TO
DATA STRUCTURES
WITH APPLICATIONS
Jean-Paul Tremblay Paul G. Sorenson
SECOND EDITION
INDIAN EDITION
FOR SALE IN INDIA ONLY

Thanks for Your Attention!

General guidelines for Solving programming assignment questions

[0] Before You Start Coding

- **Understand the Problem Statement**: The first and foremost step is to thoroughly understand the problem you're being asked to solve. If you don't understand the problem, you can't solve it. Read the problem statement multiple times and identify the key requirements.
- **Break Down the Problem**: Divide the main problem into smaller, more manageable sub-problems or tasks. This makes it easier to tackle and keeps you focused on one thing at a time.
- **Plan Your Approach**: Once you understand the smaller tasks, think about how you'll solve each one. What data structures will you need? What functions might be helpful? Sometimes, writing pseudocode can help clarify your thoughts.
- **Review Example Inputs/Outputs**: If examples are provided, examine them carefully. Try to understand how the example solutions are derived from the inputs.

General guidelines for Solving programming assignment questions

[1] While Coding

- **Start Small**: Start by solving one small part of the problem. Write just enough code to accomplish that task and test it thoroughly.
- **Iterative Development**: After the first part works, move on to the next small task. Build up your solution incrementally, testing as you go.
- **Use Version Control**: If possible, use a version control system like Git. This allows you to save working versions of your code so you can revert to them if necessary. It's like having a "save point" in a video game.
- **Comment Your Code**: Add comments to explain what each section of your code is supposed to do. This makes it easier to debug and maintain.
- **Follow Coding Standards**: Use consistent indentation, meaningful variable names, and other coding standards to make your code readable.

[2] Debugging

- **Isolate Issues**: If your code isn't working as expected, try to isolate where the issue is occurring. Comment out sections of code, print variable values to the console, or use a debugger to step through the code.
- **Check the Basics**: Before you assume that your code has a complex error, check for basic issues. Did you forget a parenthesis? Are you using the correct variable names? Did you forget to import a necessary library?

What is Debugging?

Identify Error

Identify the Error Location

Analyze Error

Prove the Analysis

Cover Lateral Damage

www.educba.com

General guidelines for Solving programming assignment questions

[3] Final Steps

- **Review Requirements**: Go back to the problem statement and ensure that your code meets all the requirements.
- **Optimize**: Once you have a working solution, think about whether you can optimize it. Can you make it run faster? Is the code clean and easy to understand?
- **Peer Review**: If possible, have a classmate or instructor review your code. They may catch errors you missed or offer suggestions for improvement.
- **Double-Check and Submit**: Finally, test your code with various test cases to ensure it's working as expected. Once you're confident that it meets the requirements, go ahead and submit it.

Problem Solving